Smart Campus Navigator

UCS 503 Software Engineering Project Report

Submitted by:

Nitin Gupta 102303525

Krish Agarwal 102303537

Akshita 102303531

BE Third Year - COE

Group: 3C41

Team Name: Beta

Submitted to: Dr. Raghav B. Venkataramaiyer

Computer Science and Engineering Department

TIET, Patiala

November **2025**

TABLE OF CONTENTS

Software Bid/ Project Teams

In the context of software development, a bid refers to a formal proposal submitted by a company or individual to a potential client, outlining the approach, timeline, cost, and team qualifications for a specific software project

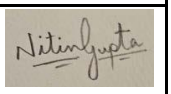
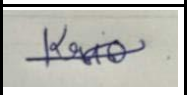
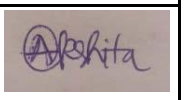
UCS 503- Software Engineering Lab

Group : 3C41 Dated: November 21,2025

Team Name: Beta
Please enter the names of your Preferred Team Members. :

· You are required to form **athreetofourperson** teams

· Choose your team members wisely. You will not be allowed to change teams.

Name	Roll No	Project Experience	Programming Language used	Signature
Nitin Gupta	102303525	Car Rental System	Js node express postgres Python	
Krish Agarwal	102303537	Car Rental System	Js node express Python	
Akshita	102303531	Restraunt Management System	Js node express postgres AI/ML	

Programming Language / Environment Experience

List the languages you are most comfortable developing in, **as a team**, in your order of preference. Many of the projects involve Java or C/C++ programming.

1.Node.js
2.postgres
3.Express.js

Choices of Projects:

Please select **4projects** your team would like to work on, by order of preference: *[Write at-least one paragraph for each choice (motivation, reason for choice, feasibility analysis, etc.)]*

First Choice	**Smart Campus Navigation & Event Finder** *This app guides users across campus via Bluetooth beacons, locates buildings, labs, and classrooms (searchable by subject), shows real-time shuttle locations, and recommends events based on interests. It leverages React/React Native with a Node.js backend, matching our team's web/mobile and real-time data skills.*	Unified indoor wayfinding, live bustracking,class/lablocator, and personalized event suggestions.
Second Choice	**Collaborative Task & Project Management Tool** *Teams assign tasks, drag cards through stages, and chat instantly with Socket.IO updates. Built on MERN (or Django/React), it fits our experience in CRUD, real-time features, and frontend frameworks.*	Integrated Kanban board, live chat,andprogressdashboards.
Third Choice	**Peer-to-Peer Marketplace** *Users list items, negotiate via chat, and complete transactions through Stripe/Razorpay. Its straightforward CRUD design and payment API integration align with our technical strengths.*	Local goods/services exchange withsecurein-apppaymentsand messaging.
Fourth Choice	**Remote Team Productivity Tracker** **Members post progress each day;** *managers view live charts and trigger Slack-style reminders for pending updates. React frontend, Node.js backend, and Chart.js visualizations ensure timely delivery.*	Daily status logs, burndown charts, and automated reminders.

AdditionalRemarks/Inputs

Please tell us about any other factors that we should take into consideration (e.g., if you really would like to work on a project for some particularly convincing reason). ……
……

……

……

1.2 Project Overview:

EXECUTIVE SUMMARY:

Smart Campus Navigator is an innovative event navigation platform designed to solve the persistent challenge of venue confusion and inefficient movement during large-scale campus events. As universities, colleges, and institutions host multiple sessions, workshops, and cultural activities across various locations, attendees often face difficulties locating venues, keeping track of schedules, and navigating unfamiliar areas. Smart Campus Navigator addresses these challenges by providing realtime venue visibility, precise navigation, and interactive event updates, ensuring a seamless experience for participants and organizers alike.

The platform combines intelligent mapping, live updates, and an intuitive interface to simplify event participation. Attendees can access an interactive digital map highlighting venues, session details, and routes, while real-time updates keep them informed about schedule changes or location modifications. Organizers can use an integrated dashboard to broadcast updates instantly, track attendee flow, and optimize resource allocation, ensuring smooth event management.

The vision of Smart Campus Navigator is to become the go-to event companion for academic, cultural, and professional gatherings, transforming how people navigate complex venues. Its core advantage lies in reducing confusion, minimizing delays, and enhancing engagement, ultimately creating an environment where participants can focus on learning, networking, and enjoying the event rather than struggling with logistics.

With its scalability and adaptability, Smart Campus Navigator has the potential to extend beyond academic campuses to corporate events, expos, and conferences, making it a versatile solution in the growing domain of smart event management.

PROPOSED SOLUTION

Smart Campus Navigator is designed as a web-based application to provide seamless event navigation and venue visibility without requiring separate mobile installations. This approach ensures maximum accessibility since attendees and organizers can access the platform directly via any browser on laptops, tablets, or smartphones, making it lightweight, fast, and universally compatible.

Core Features of the Web Application

1. **Interactive Campus Map** ○ A dynamic map integrated within the web platform, displaying all event venues in real-time.
    - Color-coded location markers and search functionality for quick navigation to specific sessions.
    - Route guidance from key points (entry gates, parking areas, information desks) to desired venues.
2. **Live Event Dashboard** ○ Centralized session details including timings, topics, and venue assignments.

- Real-time updates for schedule changes, cancellations, or venue reassignments.Push-style browser notifications to alert attendees immediately.

3. **Organizer Control Panel** ○ Enables organizers to update session data, venue availability, and live announcements in one place.
    - Provides real-time crowd insights and attendee flow metrics (optional future integration with IoT sensors).
4. **User Experience and Accessibility** ○ Optimized for multiple screen sizes (desktop, tablet, and mobile browsers).
    - Minimalistic design for easy navigation with no installation overhead.
    - Multi-language interface to cater to diverse attendee groups.

Operational Flow

- Attendees open the Smart Campus Navigator web app via a URL/QR code.
- They view a live map, session listings, and navigation suggestions directly on their browser.
- Organizers push instant updates, visible across all connected devices in real-time.

Novelty / Unique Selling Point (USP)

Smart Campus Navigator introduces a **fresh approach to event navigation** by addressing long-standing issues in campus and large-scale gatherings: **confusion about venue locations, poor schedule visibility, and lack of realtime guidance.** Unlike conventional event brochures, static maps, or bulky mobile applications, Smart Campus Navigator provides a **lightweight, browser-based platform** that is **accessible instantly** and offers **seamless redirection to trusted mapping services** for precise route guidance.

Why Smart Campus Navigator is Unique

1. **Zero Installation – Maximum Accessibility** ○ No need for users to download or install any application.
    - A simple QR scan or URL link grants instant access to event details and navigation features.
2. **Dynamic Event Dashboard with Integrated Map Redirection** ○ Combines a visually clear event dashboard with live session information.

- Redirects users to mapping platforms (e.g., Google Maps) for precise walking or driving routes – ensuring both accuracy and familiarity.

3. **Real-Time Information Updates** ○ Any change in venue, schedule, or session details is reflected immediately for all users.
   - Organizers can make updates on the fly via a central dashboard.
4. **Minimalist, Intuitive Design for Diverse Users** ○ Easy to use, even for first-time visitors or participants unfamiliar with digital tools.
   - Multi-device compatibility ensures smooth access via smartphones, tablets, or laptops.
5. **Scalable for Multiple Event Types** ○ While designed for university campuses, the framework can be extended to **corporate expos, trade fairs, cultural festivals, and government conferences**, showcasing high adaptability.
6. **Focus on Efficiency Over Complexity** ○ Instead of reinventing navigation technology, Smart Campus Navigator leverages **existing high-precision mapping services**, keeping the platform light, cost-efficient, and reliable.

Objectives

Smart Campus Navigator aims to deliver a **seamless, efficient, and technology-driven event experience** for attendees and organizers alike.

Primary Objectives

- **Enhance Event Navigation:**
  Provide real-time navigation to event venues through a simple web interface.
- **Improve Event Engagement:**
  Reduce time spent locating venues, increasing session participation.
- **Enable Real-Time Updates:**
  Ensure instant access to updated session schedules and venue details without refresh delays.
- **Optimize Crowd Flow:**
  Direct attendees to correct locations efficiently, minimizing congestion and confusion.

Deliver Universal Accessibility:

Provide a browser-based solution that requires no additional installations or device configurations.

Methodology

Smart Campus Navigator will be developed specifically for **college campuses**, where events often span multiple buildings, auditoriums, and open areas. The methodology follows a phased approach for reliability and scalability.

1. Requirement Analysis

- Identify key navigation issues faced by students and visitors during campus fests, hackathons, and academic events.
- Map all campus venues, including indoor halls, labs, seminar rooms, and open stages.
- Determine data sources for schedules and real-time updates (organizer inputs).

2. System Design

- Build a web-based platform with an **interactive dashboard** showing venues, session details, and schedules.
- Plan **Google Maps API integration** for accurate route guidance within and around campus.
- Design a simple admin panel for organizers to update session and venue details quickly.

3. Development

- **Frontend:** Responsive interface built for mobile-first use (HTML, CSS, JavaScript).
- **Backend:** Lightweight framework (Flask/Django/Node.js) to handle session data, venue details, and updates.
- **Integration:** Coordinate mapping service for redirection to Google Maps for turn-by-turn navigation.

4. Testing & Validation

- Pilot testing during smaller campus events.
- Stress testing for peak usage (hundreds or thousands of students simultaneously).
- Feedback collection to refine usability and accuracy.

5. Deployment

- Host on a secure cloud server accessible across campus Wi-Fi and mobile networks.
- Final roll-out during a major college event for real-world validation.

Project Deliverables / Outcomes Deliverables

- Web application accessible via campus Wi-Fi.
- Interactive dashboard showing:
    - Event name, session time, and venue location.

- Map redirection for navigation.

· Admin panel for organizers to update schedule and venue changes instantly.

Outcomes

· Improved attendee experience, reducing time spent searching for venues. · Higher participation in parallel sessions as navigation becomes easier.

· Reduced confusion for first-year students and visitors unfamiliar with campus layout.

Look and Feel of Product / Product Perspective

· **Interface:** Clean, intuitive, and designed for quick access.

· **Campus Map:** Markers for halls, auditoriums, and open spaces with clear labels.

· **Color Coding:** Different colors for session types (workshops, cultural shows, tech talks).

· **Device Compatibility:** Optimized for mobile browsers as most students will use smartphones.

· **Organizer View:** Admin dashboard accessible via secure login for instant updates.

Scope of Application

Smart Campus Navigator will primarily serve:

· **Campus Fests & Cultural Events** – guiding students to multiple performance and competition venues.

· **Hackathons & Technical Events** – helping participants locate workshops, coding arenas, and presentation halls.

· **Seminars & Guest Lectures** – ensuring students and faculty reach sessions on time.

· **Campus Tours for Visitors & Parents** – providing an easy guide to facilities during admission seasons or open days.

Future scalability can extend Smart Campus Navigator to **inter-college festivals**, **state-level student conventions**, and **multi-campus university networks**.

Timeline

Phase 1: Planning & Design (Weeks 1-4)

- Requirements gathering and analysis
- System design and architecture
- UI/UX design and prototyping
- Backend database schema design

Phase 2: Development (Weeks 5-12)

- print 1: Core Navigation features (GPS-based), including class finding and lecture navigator
- Sprint 2: Event Finder and database integration, with faculty office finder
- Sprint 3: Web application development
- Sprint 4: Machine Learning integration

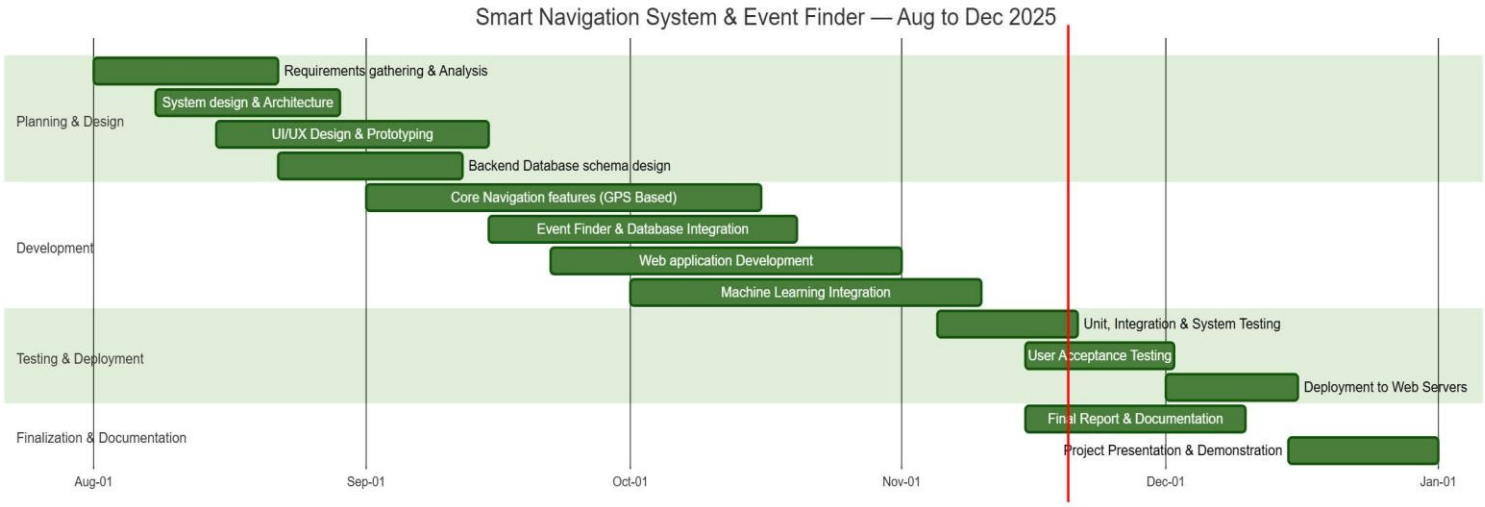
Phase 3: Testing & Deployment (Weeks 13-15)

- Unit testing, integration testing, and system testing
- User acceptance testing
- Deployment to app stores and web servers

Phase 4: Finalization & Documentation (Week 16)

- Final report and documentation Project presentation and demonstration

Gantt chart:

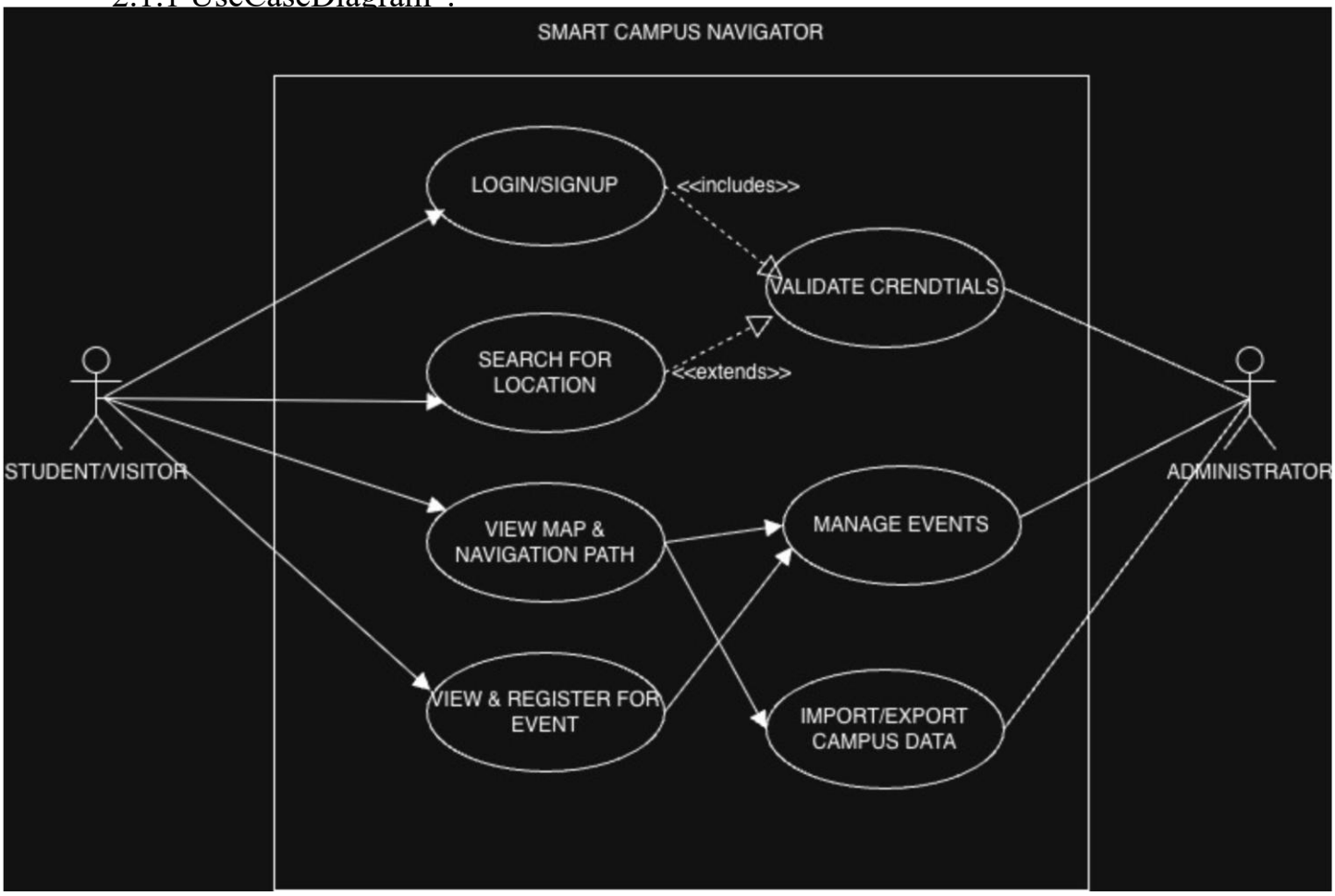


2. Analysis Phase

2.1 Use Cases:

1. Register/Login
2. View Interactive Map & Layers
3. Outdoor Navigation
4. Browse and Filter Events
5. Admin Manage Events
6. View History (Routes and Events)
7. Search Location
8. Manage Profile & Preferences

2.1.1 UseCaseDiagram :



2.1.2 Use Case Template:

Use Case Templates – Smart Navigator & Event Finder

Use Case Template – UC-01

Use Case Title	Search & Filter Events
Abbreviated Title	SEARCH
Use Case ID	UC-01
Actors	User (Visitor), Events Catalog, Search Service
Description	Allows users to discover events quickly through keyword search and filters such as date, city, category, price, and distance.
Pre-conditions	Platform is online; events exist in the catalog.
Task Sequence	User opens Events page or search bar. System displays initial results. User applies filters (date/city/category/price/distance). System refines, sorts, and paginates results. User opens an event or saves to wishlist.
Post-conditions	A filtered results list is displayed; optional wishlist entries are saved.
Modification History	V1.0 (15-Oct-2025): Initial version.
Author	Smart Navigator & Event Finder Team

Use Case Template – UC-02

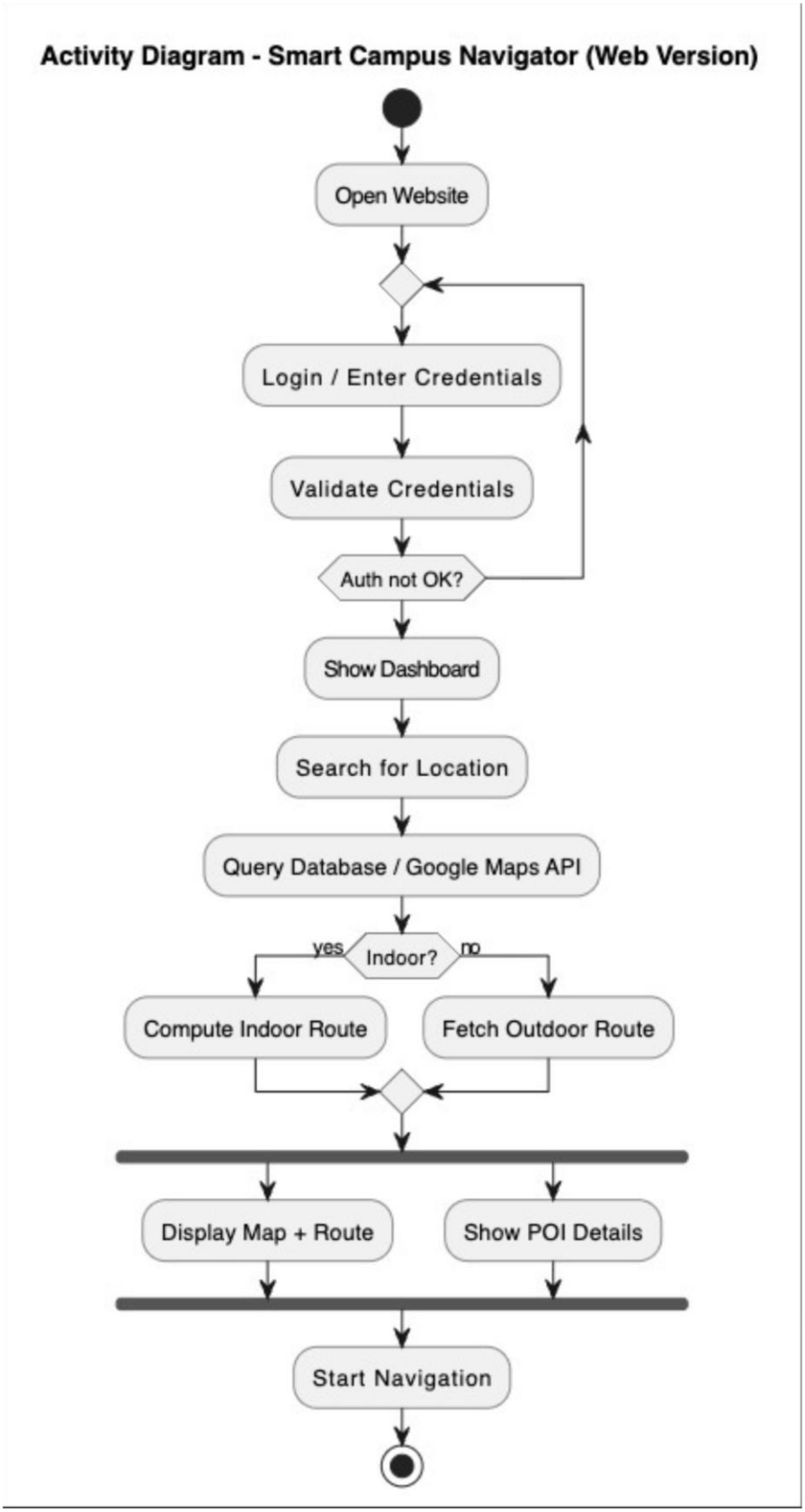
Use Case Title	Register / Buy Ticket for Event
Abbreviated Title	REGISTER
Use Case ID	UC-02
Actors	Registered User, Payment Gateway, Email/SMS Service, Ticketing Service
Description	Enables a logged-in user to purchase/register for an event and receive an e-ticket/QR code.
Pre-conditions	User is authenticated; event is open; seats available.
Task Sequence	User selects ticket type/quantity and proceeds to checkout. System validates availability and pricing (taxes/fees/promos). User enters attendee info and chooses payment method. System processes payment via Payment Gateway. On success, System generates order & ticket IDs and sends confirmation (email/SMS with QR).
Post-conditions	Order is created, payment captured, and eticket/receipt issued. On failure, no charge and inventory restored.
Modification History	V1.0 (15-Oct-2025): Initial version.
Author	Smart Navigator & Event Finder Team

Use Case Template – UC-03

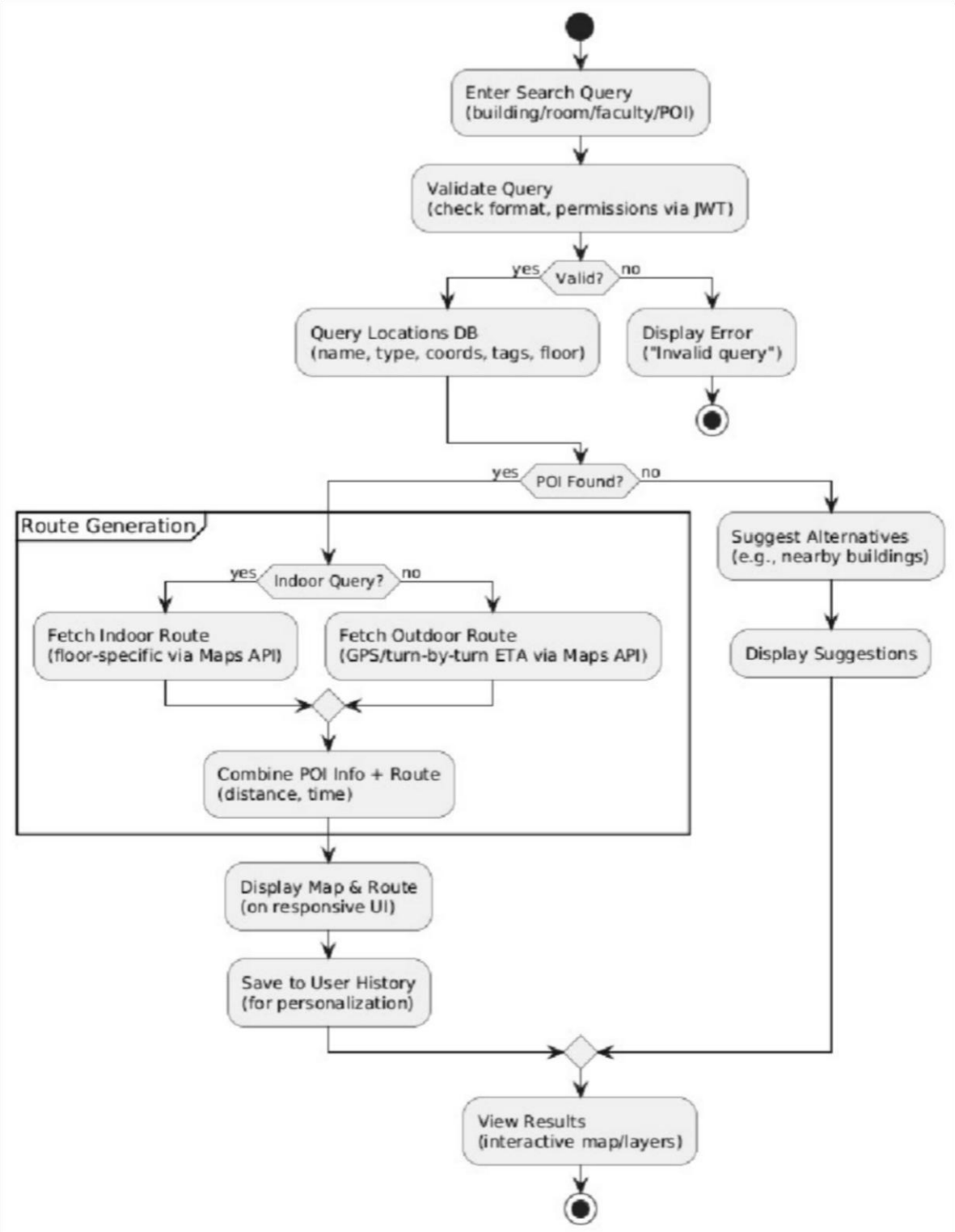
Use Case Title	Get Smart Navigation Route
Abbreviated Title	ROUTE
Use Case ID	UC-03
Actors	User (Attendee/Visitor), Maps Service, Location Service, Traffic/Transit API, Events DB
Description	Computes and presents an optimized route with ETA and guidance to the selected venue/event.
Pre-conditions	User has provided/allowed start location; network available; destination known.
Task Sequence	

	User taps 'Navigate' on an event/venue. System fetches live traffic/transit and constraints (closures). System computes candidate routes (fastest, shortest, transit-friendly). System displays routes with ETA and distance; user selects one. System starts turn-by-turn guidance; reroutes on deviations.
Post-conditions	An active route with guidance is available; fallback text directions if APIs fail.
Modification History	V1.0 (15-Oct-2025): Initial version.
Author	Smart Navigator & Event Finder Team

2.2 Activity and Swimlane Diagram

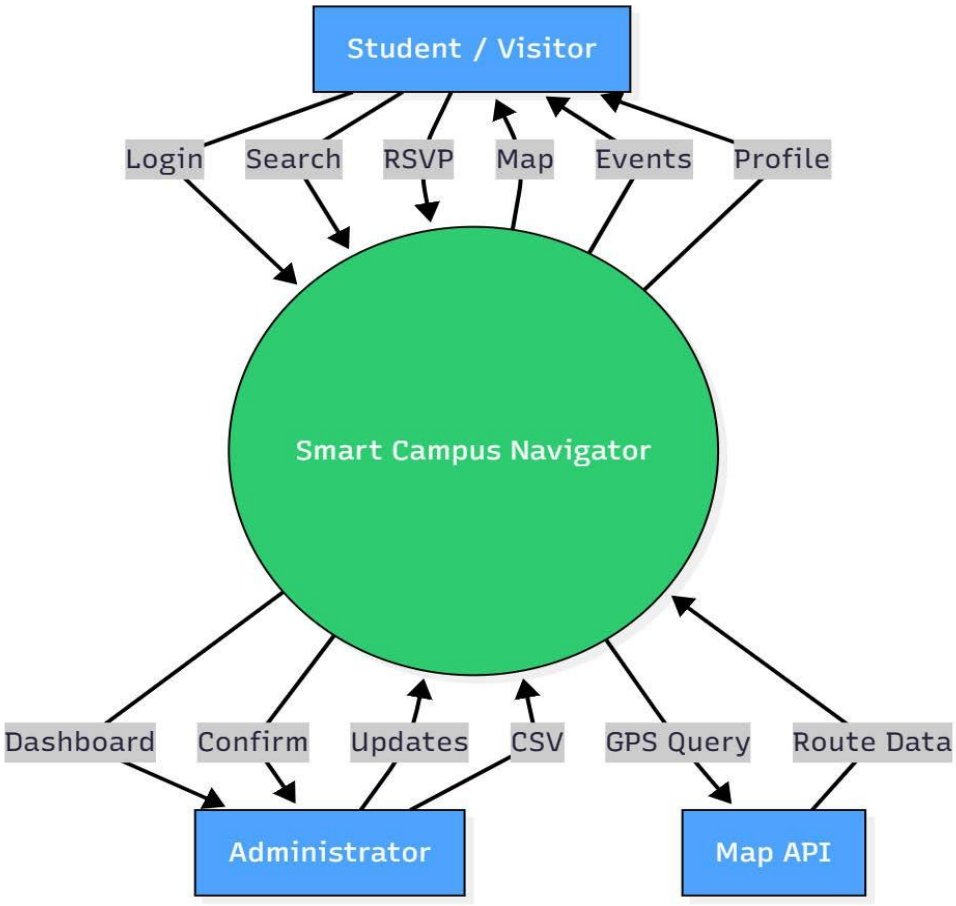


2.2.1 Navigation Activity Diagram

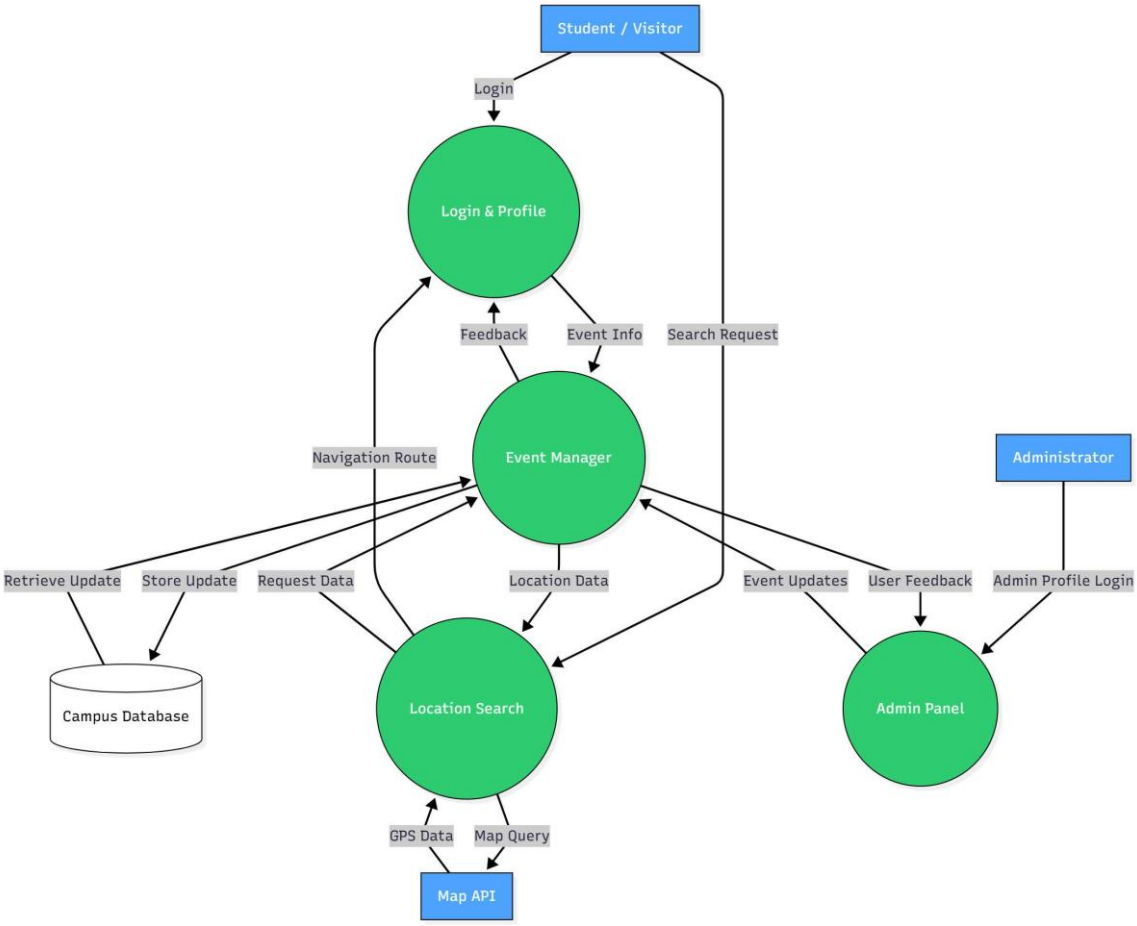


2.3 Data Flow Diagram

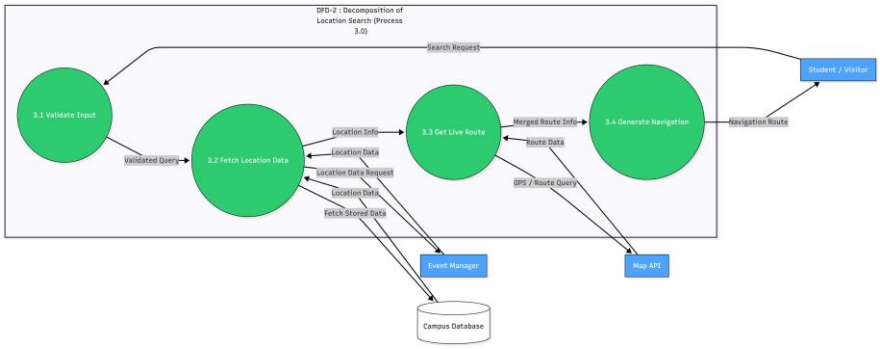
2.3.1 DFD 0

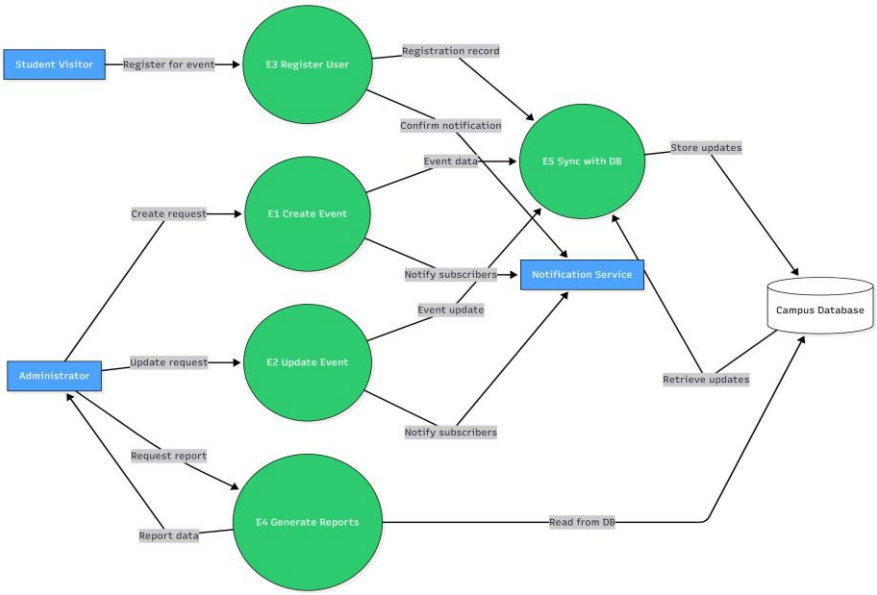


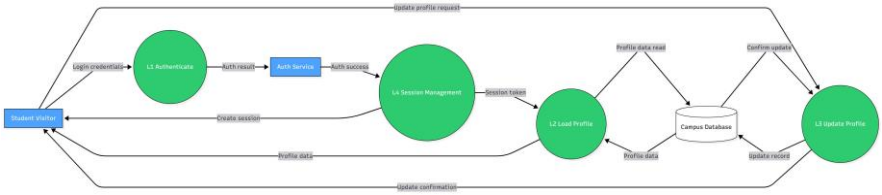
2.3.2 DFD-1

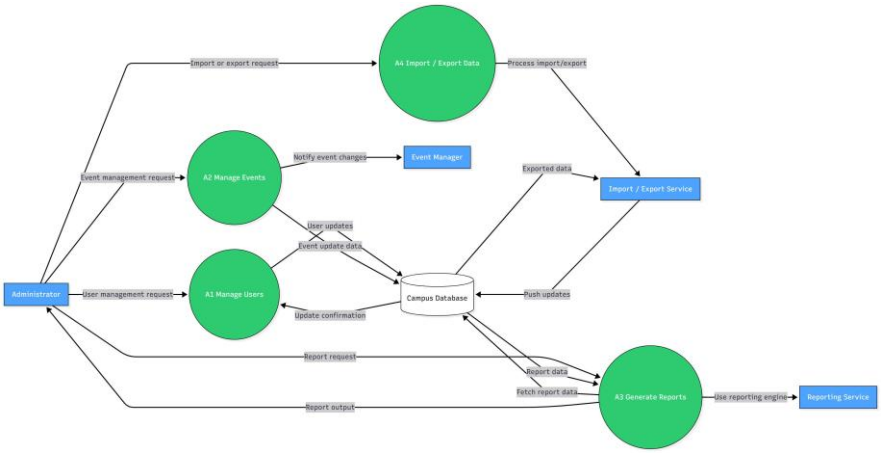

Student / Visitor
Login
Login & Profile
Feedback
Event Info
Search Request
Navigation Route
Event Manager
Administrator
Retrieve Update
Store Update
Request Data
Location Data
Event Updates
User Feedback
Admin Profile Login
Campus Database
Location Search
Admin Panel
GPS Data
Map Query
Map API

2.3.3 DFD-2


DFD-2 : Decomposition of Location Search (Process 3.0)
Search Request
Student / Visitor
3.1 Validate Input
Validated Query
3.2 Fetch Location Data
Location Info
Location Data
Location Data Request
Location Data
Fetch Stored Data
3.3 Get Live Route
Merged Route Info
Route Data
GPS / Route Query
3.4 Generate Navigation
Navigation Route
Event Manager
Map API
Campus Database


Student Visitor
Register for event
E3 Register User
Registration record
Confirm notification
E5 Sync with DB
Store updates
Event data
Create request
E1 Create Event
Notify subscribers
Notification Service
Campus Database
Event update
Administrator
Update request
E2 Update Event
Retrieve updates
Notify subscribers
Request report
E4 Generate Reports
Report data
Read from DB


Update profile request
Login credentials
L1 Authenticate
Auth result
Auth Service
Auth success
L4 Session Management
Session token
Profile data read
L2 Load Profile
Campus Database
Confirm update
L3 Update Profile
Student visitor
Create session
Profile data
Profile data
Update record
Update confirmation


Import or export request
A4 Import / Export Data
Process import/export
Event management request
A2 Manage Events
Notify event changes
Event Manager
Exported data
Import / Export Service
User updates
Event update data
Administrator
User management request
A1 Manage Users
Campus Database
Push updates
Update confirmation
Report request
Report data
Fetch report data
A3 Generate Reports
Use reporting engine
Reporting Service
Report output

2.4 SRS

Software Requirements Specification Document

Group 3C41 - Team Beta

Smart Navigator & Event Finder(Smart Campus Navigator)

TABLE OF CONTENTS

1. Introduction

1.1 Purpose of this Document

This Software Requirements Specification (SRS) document provides a detailed description of the "Smart Navigator & Event Finder" web application. It outlines the functional and non-functional requirements, system constraints, and interface specifications. This document is intended to serve as a reference for developers, testers, project managers, and stakeholders to ensure a common understanding of the project's goals and deliverables.

1.2 Scope of the Development Project

The goal is to design and develop a comprehensive web application to overcome the challenges associated with campus navigation and event discovery. The system aims to deliver intelligent, seamless, and personalized services for students, faculty members, and visitors by integrating location-based services, real-time navigation assistance, and event management within a single digital platform.

This project focuses on bridging the gap between physical infrastructure and digital convenience, ensuring that every user can easily locate facilities, discover upcoming events, and plan their schedules efficiently.

The software must be capable of performing the following operations:

1. Campus Navigation:
   The system must provide accurate, turn-by-turn directions to all Points of Interest (POIs) on the campus. These POIs include classrooms, laboratories, lecture halls, faculty offices, auditoriums, libraries, hostels, cafeterias, sports complexes, and administrative buildings.

2. Outdoor Navigation:
   The system must utilize GPS and mapping data to provide clear guidance between buildings, ensuring users can navigate the campus effectively, even during peak hours or while attending back-to-back classes in different locations.

3. Indoor Navigation:
   The system must provide navigation support within multi-floor buildings where traditional GPS signals may not work. Indoor navigation should include:

   - Class Finding: Locate and provide precise routes to scheduled classrooms, including floor details and nearest entry points.

   - Faculty Office Finder: Assist students and visitors in locating faculty and staff offices efficiently, reducing time spent searching in large academic complexes.

   - Lecture Navigator: Guide users to lecture halls, auditoriums, and seminar rooms, ensuring timely arrival for large gatherings or academic sessions.

4. Event Management and Discovery:
   The system must serve as a centralized platform for displaying all campus events, ranging from academic seminars and workshops to cultural activities and sports events.

   - Personalized Recommendations: Advanced machine learning algorithms will analyze user preferences, navigation history, and event participation patterns to recommend relevant events.

   - Event Details: Each event listing will include time, date, location, organizer details, and a briefcomplete information before attending.

   - Dynamic Filtering and Notifications: Users should be able to filter events by category (academic, cultural, sports, etc.) and receive realtime notifications for upcoming events.

5. User Profile Management:
   The system must allow users to create personal profiles where they can:

   - Set navigation preferences (e.g., shortest route, least crowded path).

   - Save frequently visited locations and favorite events.

- View their navigation history, past event attendance, and upcoming schedules.

The web application will be centrally hosted on a secure web server, and all data—including spatial information, event listings, and user profiles—will be stored in a robust backend database. The database will be designed to support real-time updates and ensure data consistency across multiple modules.

Initially, the system will be implemented for the main university campus as part of a Pilot Phase, targeting key facilities and a primary user group of students, faculty, and administrative staff. Once the Pilot Phase proves successful, the project will be scaled to cover the entire campus and potentially extended to other campuses of the institution.

In the future, this system can be adapted for broader applications beyond educational institutions. Large-scale environments such as corporate parks, industrial

S.No	Term	Definition
1	API	Application Programming Interface. A set of rules for building and interacting with software applications.
2	GPS	Global Positioning System. A satellite-based radio navigation system.
3	ML	Machine Learning. A field of artificial intelligence that uses algorithms to learn from and make predictions on data.
4	POI	Point of Interest. A specific physical location that someone may find useful or interesting.
5	UI	User Interface. The means by which the user and a computer system interact.
6	UX	User Experience. A person's emotions and attitudes about using a particular product, system, or service.
7	Geofencing	A virtual boundary defined using GPS or RFID technology to trigger actions when a device enters or leaves a location.

8	Responsive Design	A design approach that ensures web applications adjust smoothly across different devices and screen sizes.
9	Cloud Hosting	Hosting of applications and data on remote servers accessed via the internet to ensure scalability and high availability.
10	Real-Time Database	A database that updates information instantly across all connected devices and users.

complexes, research facilities, and public infrastructure hubs (e.g., convention centers, hospitals, or airports) can benefit from similar navigation and event management solutions. The modular architecture of the system ensures scalability and ease of customization, making it a versatile platform for multiple domains.

1.3 Definitions,abbreviationsandacronyms

Table 1: Definitions

11	Pathfinding Algorithm	An algorithm used to determine the shortest or most efficient route between two points in a navigation system.
12	Data Caching	Temporary storage of data to improve application performance and reduce server load.
13	Role-Based Access Control (RBAC)	A method of restricting system access based on the roles of individual users.
14	Session Management	A technique for managing user interactions across multiple requests, ensuring secure and consistent experiences.
15	Encryption	The process of encoding data to protect it from unauthorized access.

Table 2: Abbreviations

S.No	Mnemonic	Full Form
1	SRS	Software Requirements Specification
2	GIS	Geographic Information System
3	HTTPS	Hypertext Transfer Protocol Secure
4	JSON	JavaScript Object Notation
5	OS	Operating System
6	MVC	Model-View-Controller
7	REST	Representational State Transfer
8	CI/CD	Continuous Integration / Continuous Deployment
9	UML	Unified Modeling Language
10	SDK	Software Development Kit

1.4 References

[1]. Agile Methodology Overview. Link: https://www.agilealliance.org/agile101/

[2]. REST API Concepts. Link: https://restfulapi.net/

[3]. Microservices Architecture Guide. Link: https://martinfowler.com/articles/microservices.html

[4]. CI/CD Pipeline Explained. Link: https://www.redhat.com/en/topics/devops/whatis-ci-cd

[5]. Containerization Using Docker. Link: https://www.docker.com/resources/whatcontainer

[6]. UML Basics. Link: https://www.uml-diagrams.org/

[7]. Version Control Systems (Git). Link: https://git-scm.com/book/en/v2

[8]. Software Development Life Cycle (SDLC) Overview. Link: https://www.softwaretestinghelp.com/software-development-life-cycle-sdlc/

[9]. Test-Driven Development (TDD) Approach. Link: https://www.ibm.com/topics/test-driven-development

[10]. Cloud Computing Models: IaaS, PaaS, SaaS. Link: https://azure.microsoft.com/en-us/overview/what-is-cloud-computing/

[11]. Load Balancer Concepts. Link: https://www.nginx.com/resources/glossary/loadbalancing/

12]. DevOps Practices and Tools. Link: https://www.atlassian.com/devops

[13]. Software Refactoring Techniques. Link: https://refactoring.guru/refactoring

1.5 Overview

The remainder of this document is structured as follows: Section 2 provides an overall description of the product, its functions, users, and operating constraints. Section 3 contains the detailed, specific requirements of the system, including functional, interface, performance, and database requirements. This SRS is comprehensive and is the foundation for the design, implementation, and testing phases.

2. Overall Description

2.1 Product Perspective

The "Smart Navigator & Event Finder" is a responsive web application designed to function as a comprehensive digital guide to the college campus. It operates directly within the web browser of a user's standard device, such as a smartphone or laptop, requiring no specialized software installation. The primary user interface is an interactive campus map, which users navigate with intuitive on-screen controls and touch gestures, such as tapping on buildings or using a search bar to find locations.

For instance, a new student trying to find their orientation seminar can open the application, type "Main Auditorium" into the search bar, and the system will instantly highlight the building on the map. With another tap, the application can generate the most efficient walking route from the student's current position to the auditorium entrance, complete with an estimated travel time.

The underlying mechanism that makes this possible is best explained by examining the system's four major software modules, as depicted in the block diagram below (Figure 1).

Figure 1: Block Diagram of the Smart Navigator & Event Finder System The

system is comprised of four primary modules that work in sequence:

1. Map & UI Module
2. Navigation Engine
3. Data Service Module
4. User Profile Module

Upon launching the application, the user interacts directly with the Map & UI Module. This module is responsible for rendering the visual interface, including the campus map, search bars, and information panels. It fetches base map imagery from an external service (like the Google Maps API) and overlays it with our custom, detailed campus data, such as building footprints, pathways, and points of interest.

When a user requests directions, the UI Module passes the start and end coordinates to the Navigation Engine. This core backend module contains the logic for all pathfinding. It processes the campus's unique geospatial data—including all designated walkways, shortcuts, and accessibility routes—to calculate the optimal

path. This computed route is then sent back to the UI Module to be drawn on the user's screen.

To find information about a specific location or event, the UI Module communicates with the Data Service Module. This module acts as the gateway to the central Campus Database (a MongoDB instance). We can define any request for information as a "transaction." For example, a student searching for "Physics Lab 101" initiates a 'read' transaction, where the Data Service queries the database and returns the lab's location, hours, and any scheduled classes. Conversely, an administrator updating an event's time via a secure portal would be performing a 'write' transaction.

Finally, the User Profile Module manages personalization and authentication. When a user logs in, this module retrieves their saved preferences, such as "favorite" locations or registered events, from the Campus Database, creating a tailored experience.

These four modules are enclosed within the application's backend server to signify that they form the core of the system's software. All communication with the central database is handled exclusively by the backend modules, ensuring that data is processed efficiently and securely before being sent to the user's device for display.

2.2 Product Functions

The "Smart Navigator & Event Finder" software must be able to perform the following operations:

1. **User Authentication:** The system must be able to authenticate a user by verifying their credentials (e.g., university ID and password) against the user records stored in the central database. It must grant access to personalized features upon successful authentication.

2. **Profile and Preference Management:** The system must allow an authenticated user to create and manage their personal profile. This includes updating their information and setting preferences, such as event categories of interest and accessibility needs, which will be written to and stored in the database.

3. **Interactive Map Rendering:** The system must render a high-fidelity, interactive digital map of the campus. It must be able to dynamically overlay custom data layers fetched from the database, including building outlines, room numbers, points of interest, and real-time user location.

4. **Geospatial Data Querying:** The system must be able to perform search operations by querying the database for specific locations (e.g., buildings, labs, offices). Upon a successful query, it must retrieve the location's coordinates and associated details and visually pinpoint it on the map for the user.

5. **Route Calculation:** The system must be able to calculate and display the most efficient walking route between any two user-selected points on campus. This operation involves processing the stored pathway data to generate turn-byturn directions and an estimated travel time.

6. **Event Data Retrieval and Filtering:** The system must be able to query the database to retrieve a comprehensive list of all campus events. It must provide functionality for any user to filter this data based on various attributes, such as date, location, and event category (e.g., academic, social, sports).

7. **Content Management for Administrators:** The system must provide a secure administrative interface, accessible only to authorized staff. Through this panel, authorized users must be able to perform write, update, and delete operations on the event and location records stored in the central database, ensuring the application's data remains current.

2.3 User Characteristics

The "Smart Navigator & Event Finder" is designed for a diverse population within the college community, each with varying needs and technical familiarity. The system is designed to be intuitive for all users, who can be categorized into the following groups:

1. Students: The primary audience, including undergraduate and graduatestudents. Their main use will be for navigating to classes, finding

campus services (like the library or dining halls), and discovering social or academic events.

2. Faculty & Staff: Academic and administrative personnel who will use the systemto find meeting rooms, colleagues' offices, lecture halls, and stay informed about university-wide events and schedules.(We will try to update teacher’s cabin number along with their respective mails and cabin number)

3. Campus Visitors: A broad category that includes prospective students and theirparents, alumni, guest speakers, and conference attendees. These users require temporary, easy-to-access guidance around an unfamiliar campus.

4. System Administrators: A limited group of authorized personnel (e.g., from theIT or Events department) responsible for managing the application's content. Their interaction will be through a dedicated administrative panel to update map data and event information.

2.4 General Constraints, Assumptions and Dependencies

The following list presents the constraints, assumptions, dependencies or guidelines that are imposed upon implementation of the Smart Navigator & Event Finder software:

Our app is a website, not something you download from an app store. This means it has to work well on all the popular, up-to-date web browsers like Chrome, Firefox, and Safari.

A stable internet connection is absolutely necessary. Since all the map and event info is live, the app won't be able to do much if you're ofline.

The app is only as good as the information we put into it. Its usefulness completely depends on the college admins keeping event schedules and room numbers accurate and current.

We're not building the map from scratch; we're relying on an external service like Google Maps. This is a big dependency, so if their service has an outage or changes its rules, our app will be directly affected.

Nobody likes a slow app, so we're aiming to make sure that loading the map or getting directions happens within just a few seconds.

We're assuming that users will have a reasonably modern smartphone or laptop with GPS enabled for the navigation features to work properly.

The faculty office locator feature introduces a critical data dependency. Its accuracy is constrained by the availability of a centralized, official source for faculty office assignments. Furthermore, this data is highly volatile due to real-world staff and office changes, creating a significant risk of the application providing outdated information if not continuously synchronized.

2.5 Apportioning of requirements

The project will be delivered in phases aligned with the Agile methodology:

Phase 1 (MVP): Core outdoor navigation, basic event listing, and user authentication.

Phase 2: Implementation of indoor navigation features (Class Finder, Office Finder, Lecture Navigator).

Phase 3: Integration of the machine learning model for personalized event recommendations and advanced administrative features.

3. Specific Requirements

3.1 External Interface Requirements

The external interface requirements define how the system interacts with users, hardware, software, and communication protocols. These requirements ensure usability, security, and compatibility across devices.

User Interface: The UI shall be a responsive web application compatible with major browsers (Chrome, Firefox, Safari, Edge). It shall use consistent iconography, intuitive

typography, and a color-blind friendly palette. Accessibility standards (WCAG 2.1) must be followed to ensure usability for differently abled users.

Hardware Interface: The application shall be accessible on smartphones, tablets, laptops, and desktop computers with a modern web browser and stable internet connection. GPS functionality is required for outdoor navigation precision, while Wi-Fi-based triangulation may be used indoors.

Software Interface: The backend server shall communicate with the frontend via a RESTful JSON API. The system shall also support WebSockets for real-time updates and integrate with external APIs such as Google Maps and university event systems.

Communications Interface: All communications between client and server shall be encrypted using HTTPS. SSL certificates must be maintained and renewed on time. The system should support at least TLS 1.3 for encryption standards.

3.2 Detailed Description of Functional Requirements

Table 3: Template for describing functional requirements

Purpose	A description of the functional requirements and its reasons
Inputs	What are the inputs; in what form will they arrive; from what sources.
Processing	Describes the outcome; includes validity checks, timing, error handling
Outputs	The form, destination, and volume of output; error messages.

3.2.1 Functional Requirements for Navigation Module (FR-Nav-01)

Table 4: Search for a Location

Purpose	To allow the user to find and navigate to a specific point of interest (POI) on campus, ensuring accuracy, ease of use, and multiple path options.
Inputs	User types a search query (e.g., 'Main Library', 'Dr. Smith's Office'). The query may include keywords, abbreviations, or categories. Optional filters include building type, accessibility options, and shortest/fastest path preference.
Processing	System queries the database for POIs matching the search string. Natural language processing improves query handling. Geospatial algorithms
	calculate multiple routes (shortest, accessible, fastest). The system checks real-time constraints such as blocked paths or building closures.
Outputs	A ranked list of POIs is displayed, each with a description, accessibility notes, and distance. Selecting a POI centers the map, highlights the route, and provides step-by-step navigation instructions.

Purpose	To allow the user to find and navigate to a specific point of interest (POI) on campus, ensuring accuracy, ease of use, and multiple path options.

3.2.2 Functional Requirements for Event Finder Module (FR-EVT-01)

Table 5: Get Event Recommendations

Purpose	To proactively show the user events they are likely to be interested in and enhance campus engagement.
Inputs	Implicit input: User's past event attendance, profile preferences, and saved events. Explicit input: User searches for specific events or applies filters like date, category, and location.
Processing	A machine learning model processes user data, event metadata, and collaborative filtering. It considers popularity, similarity to past choices,
	and relevance to academic or personal interests. The system continuously learns from feedback such as event attendance or skipped suggestions.
Outputs	A personalized dashboard displays recommended events with title, description, time, venue, and RSVP option. Event reminders are sent via notifications or email. The system highlights trending events across campus.

3.3 Performance requirements

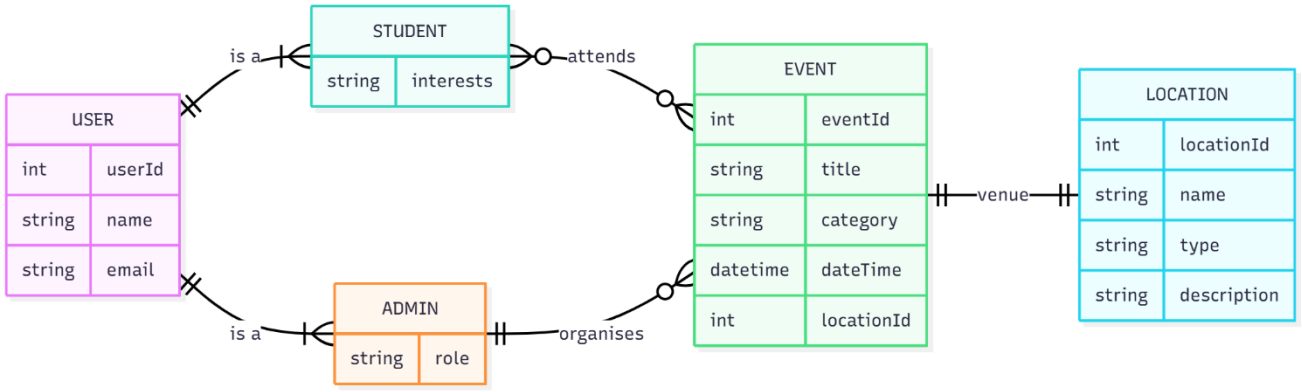
Requirement	Specification
Dashboard Load Time	The dashboard shall load in less than 3 seconds on broadband and under 5 seconds on 4G/5G mobile networks.
Search Response Time	Search results must appear within 2 seconds for queries under 5000 records; under 5 seconds for larger queries.
Route Calculation	Navigation routes generated within 5 seconds, factoring in accessibility constraints and detours.
Concurrent Users	System must support 100 concurrent users during peak load; scalable up to 500 with load balancing.
Availability	System must maintain 99.5% uptime with automatic failover to backup servers.

3.4 Logical database requirements

The database shall store entities including:

Entity	Attributes
Users	user_id, name, email (hashed password), preferences, history, last login time, account status, multi-factor authentication settings
Locations	location_id, name, type (building, room, office), coordinates, floor plan data, accessibility features, capacity, opening hours, description
Events	event_id, title, description, datetime, location_id, category, organizer, RSVP list, maximum capacity, tags, recurring flag
User-Event Interactions	user_id, event_id, interaction_type (viewed, attended, saved, shared), timestamp, feedback rating

ER Diagram



3.5 Quality attributes

Attribute	Specification
Usability	The interface must be intuitive for firsttime users with minimal training. It shall provide tutorials, tooltips, and consistent design patterns.
Reliability	System must have 99.5% uptime. Regular database backups every 24 hours with disaster recovery drills.
Security	Passwords must be hashed with bcrypt. Sensitive data encrypted at rest and in transit. System must comply with GDPR and data privacy regulations.
Maintainability	Code must follow standard conventions, include inline documentation, and maintain modular

	structure. Unit tests must cover at least 80% of functionality.
Scalability	The system architecture shall allow horizontal scaling with cloud infrastructure support. Performance should not degrade with user growth.

3.6 Other requirements

None at this time.

4. Change History

Version	Date	Description
1.0	2-Sep-2025	Initial Release of SRS.

5. Document Approvers

This SRS for the Smart Navigator & Event Finder system (Smart Campus Navigator)is approved by:

Dr.Raghav B. Venkataramaiyer

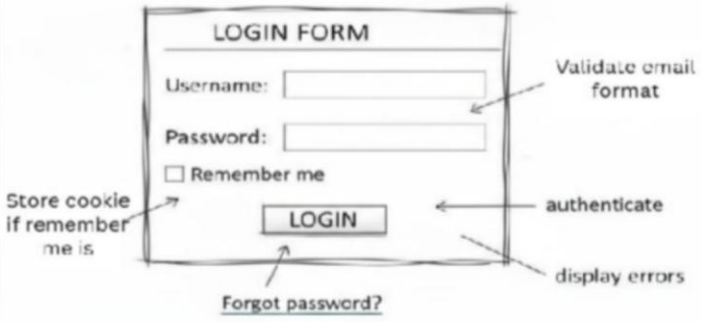
(ProjectGuide)
Date:

2.5 User story and Cards

USER LOGIN ANALYSIS

#0003 HEALTH-SMART ANALYSIS

As a registered user, I want to **start by** log in, so can access health analysis features.



Acceptance Criteria

- secure login
- supports password reset credentials
- supports remember me quick response time

Further information attached on Confluence Wiki

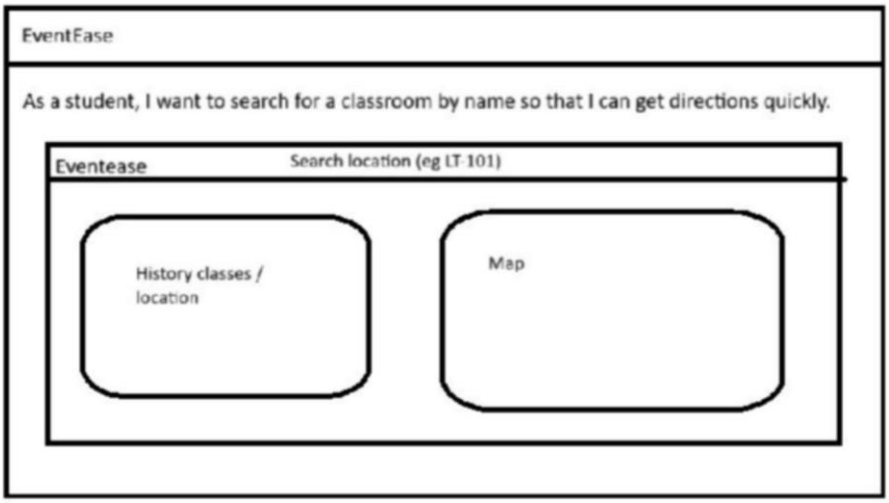
User Story: Login Page - Back of Card

Confirmation

1. **Success:**
   a. Valid credentials, user logged in and **redirected** to dashboard. **authenticated login.**
   b. Remember me stored cookie for next try login.
   c. Password reset email sent when **requested.**

2. **Failure – display message:**
   a) "Username or password incorrect.
   b) "Email format invalid
   d) Account locked after multiple failed attempts
   e) "Service unavailable
   f) Password reset failed

User stories Front Page



User stories Back of Card:

Confirmation

1.Valid Scenarios:

1. User enters valid classroom name (e.g., "A-101"); system queries Locations DB (name/type/coords/floor) and returns matching POI.
2. Route integrates user location (GPS-enabled device); saves search to history for personalization.

2. Invalid Scenarios:

1. Invalid query (e.g., misspelled "A-10X" or non-existent room); system suggests alternatives (e.g., fuzzy match nearby POIs) and displays friendly error without crashing.
2. No GPS/Internet; prompt to enable location services and fallback to manual entry.
3. Unauthorized access (pre-login); redirect to authentication

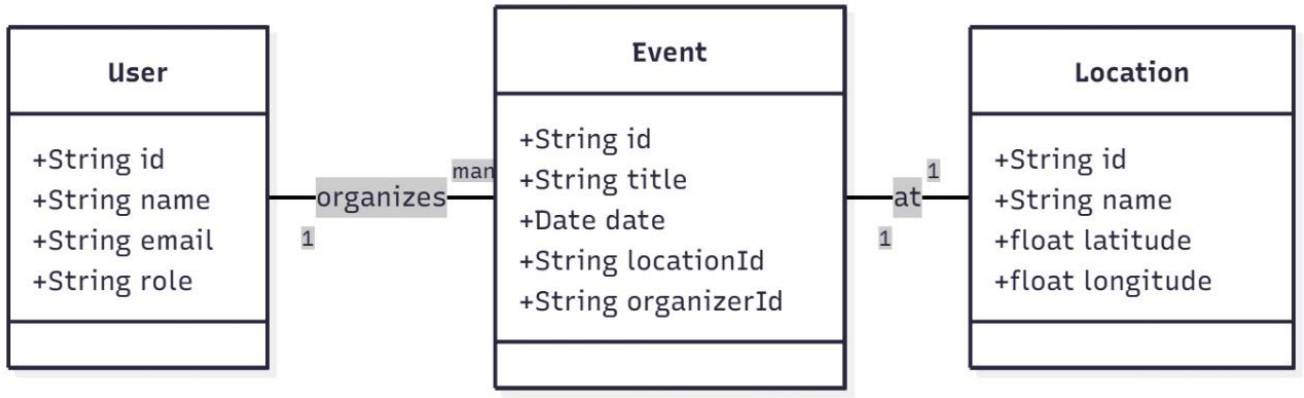
3. Design Phase

3.1 Class Diagram

Class diagrams provide a structural overview of the main entities in the project, such as User, Event, and Location, along with their attributes and relationships. In SmartNav, these diagrams help visualize how users, events, and locations are interconnected, clarifying the data model and making it easier to design and maintain the database and backend logic. By mapping out these relationships, class diagrams ensure that the system's core data structure is well understood and logically organized.

example:

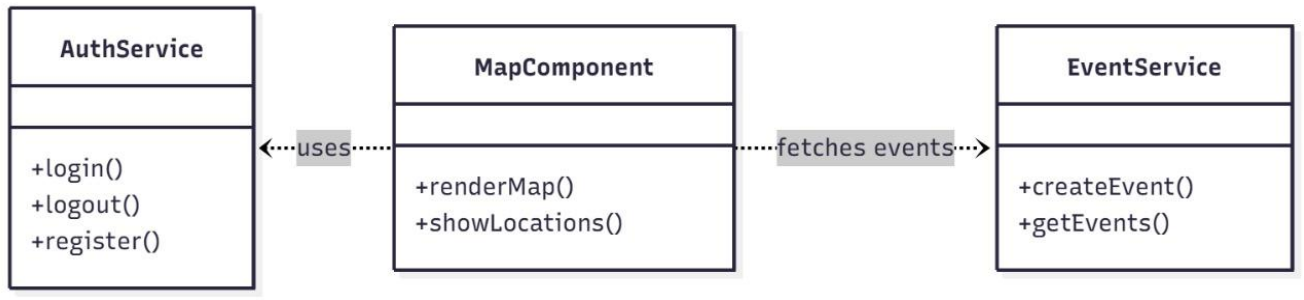
Case 1 : Based on User ,Event ,Location



Explanation:

- Shows three main entities: User, Event, Location.
- User organizes Event; Event happens at Location.
- Each class lists its attributes (e.g., User has id, name, email, role).
- Shows how data is structured and related in your system.

Case 2: Based on Auth ,Map and Event Service



Explanation:

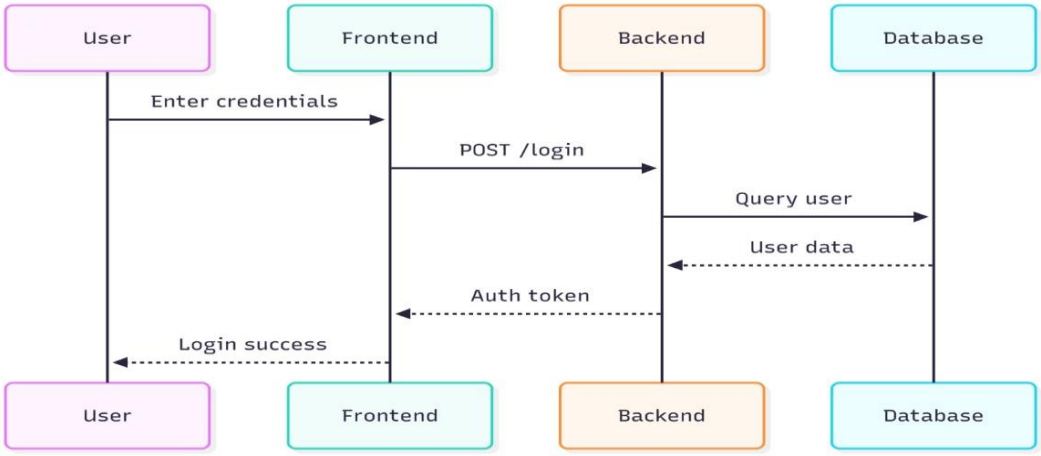
- Shows how AuthService, MapComponent, and EventService interact.
- MapComponent uses AuthService for login/logout/register, and fetches events from EventService.
- Illustrates the collaboration between frontend services/components.

3.2 Sequence Diagram

Sequence diagrams illustrate the step-by-step flow of interactions between different components or actors in the system for specific use cases. For example, they show how a user logs in or how an event is created, detailing the messages exchanged between the frontend, backend, and database. These diagrams are essential for understanding the dynamic behaviour of the system, helping developers and stakeholders follow the exact process flow and identify potential issues or improvements in the interaction logic.

Example:

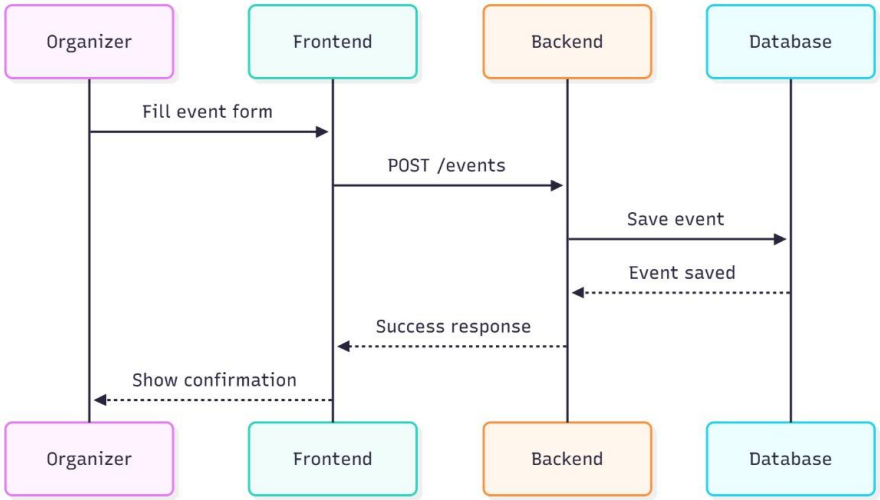
Case 1: User-Login



Explanation:

- Shows the login process.
- User enters credentials → Frontend sends to Backend → Backend queries Database → Auth token returned → User logged in.
- Visualizes the step-by-step flow for authentication.

Case 2: Create Event



Explanation:

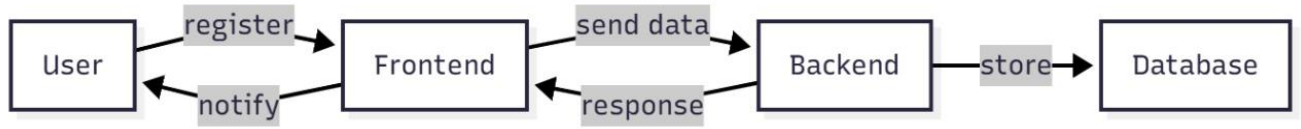
- Shows the event creation process.
- Organizer fills form → Frontend sends to Backend → Backend saves to Database → Confirmation sent back.
- Step-by-step flow for creating an event.

3.3 Collaboration Diagram

Collaboration diagrams focus on how different objects or components work together to accomplish a task. They highlight the relationships and communication paths between services, such as how the MapComponent interacts with AuthService and EventService. In the context of SmartNav, collaboration diagrams clarify the responsibilities of each component and how they cooperate to deliver features like user registration or event searching, supporting a modular and maintainable system design.

Example:

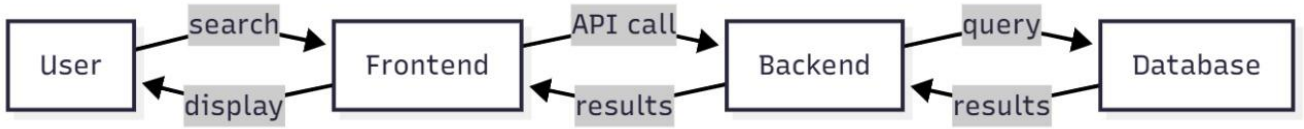
Case 1: User Registration



Explanation:

- Shows the registration process.
- User registers via Frontend, which sends data to Backend, which stores it in Database, and notifies the User.
- Focuses on how different parts work together for registration.

Case 2: Event Search



Explanation:

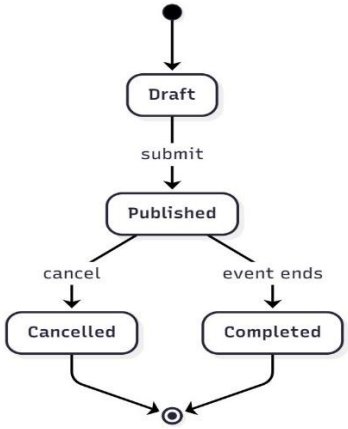
- Shows the search process.
- User searches via Frontend, which calls Backend, which queries Database, and results are displayed to User.
- Focuses on the collaboration for searching and displaying results.

3.4 State-Chart Diagram

State chart diagrams depict the various states an object can be in and the transitions between those states based on events or actions. For instance, they show how an event moves from draft to published, and then to either completed or cancelled, or how a user transitions from registered to verified or banned. These diagrams are crucial for modeling the lifecycle of key entities, ensuring that all possible states and transitions are accounted for, which helps in implementing robust business logic and user experience.

Example:

Case 1: Event State



Explanation:

- Shows the lifecycle of an Event.
- Event starts as Draft → Published → (can become) Cancelled or Completed.
- Visualizes how an event changes state over time.

Case 2: User State

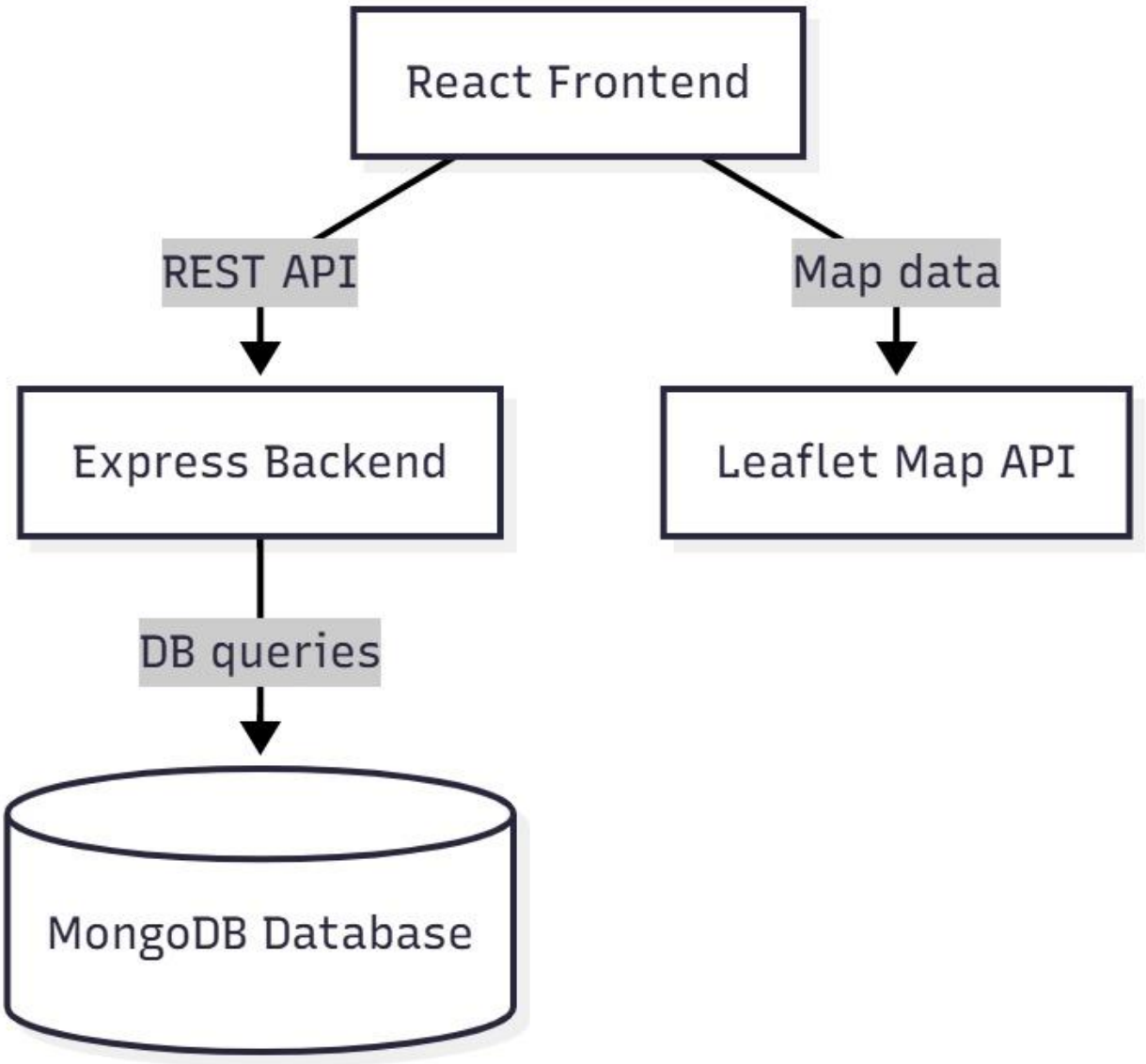
Explanation:

- Shows the lifecycle of a User.
- User is Registered → (after email confirmation) Verified → (can be) Banned by admin.
- Visualizes user status changes.

4. Implementation

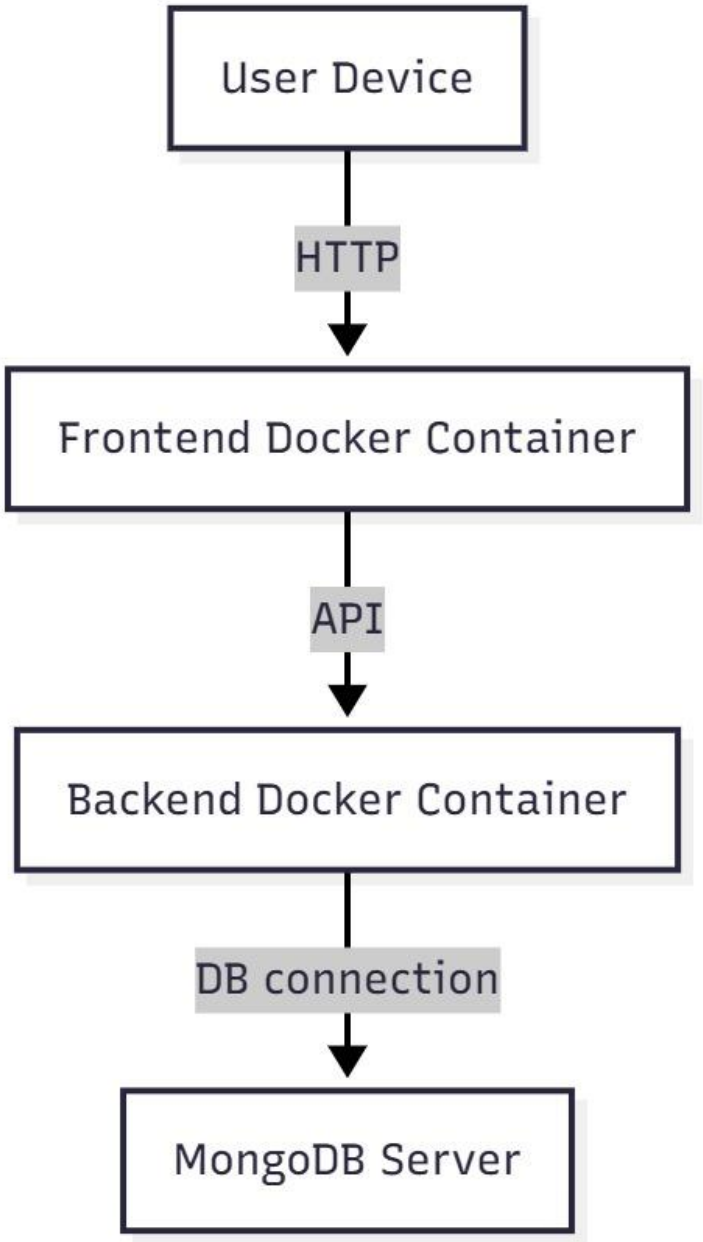
4.1 Component Diagram

Component diagrams illustrate the organization and relationships between the major software components in the system. In the context of SmartNav, the component diagram shows how the React frontend interacts with both the Express backend and the Leaflet Map API. The frontend communicates with the backend through REST API calls to fetch or update data, while it uses the Leaflet Map API to display and manage map data. The backend, in turn, handles database queries and interacts with the MongoDB database to store and retrieve information. This diagram helps clarify the responsibilities of each component and how they collaborate to deliver the application's features, making it easier to understand the system's modular structure and data flow.



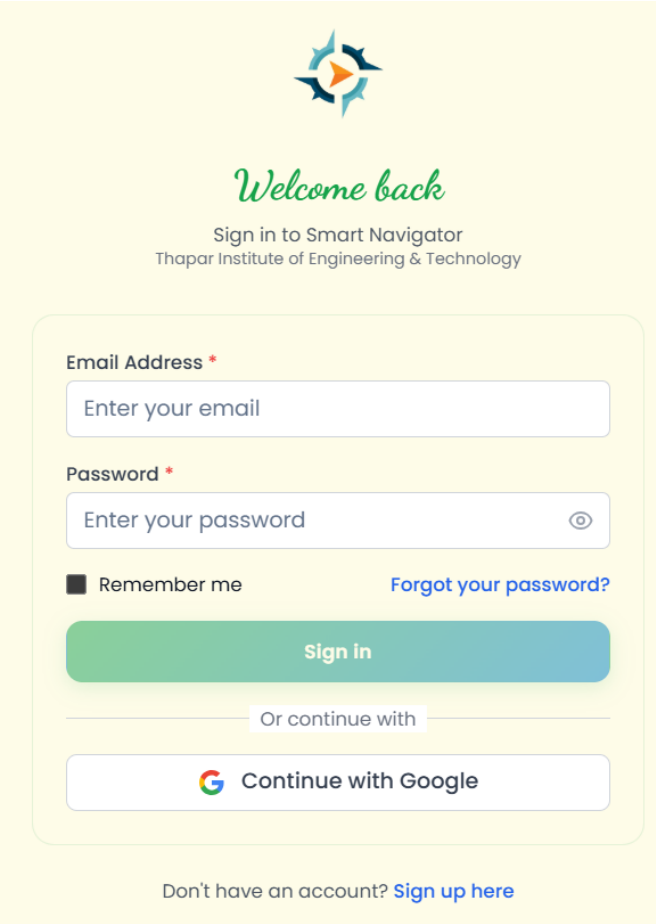
4.2 Deployment Diagram

Deployment diagrams depict the physical arrangement of hardware and software in the system, showing how different parts of the application are deployed and interact in a real-world environment. For SmartNav, the deployment diagram demonstrates how the user device connects via HTTP to a frontend Docker container, which then communicates with a backend Docker container through API calls. The backend container manages the connection to the MongoDB server for data storage and retrieval. This diagram is essential for visualizing the deployment architecture, highlighting the use of containerization (Docker) for both frontend and backend, and showing how the system components are distributed and interact in production.

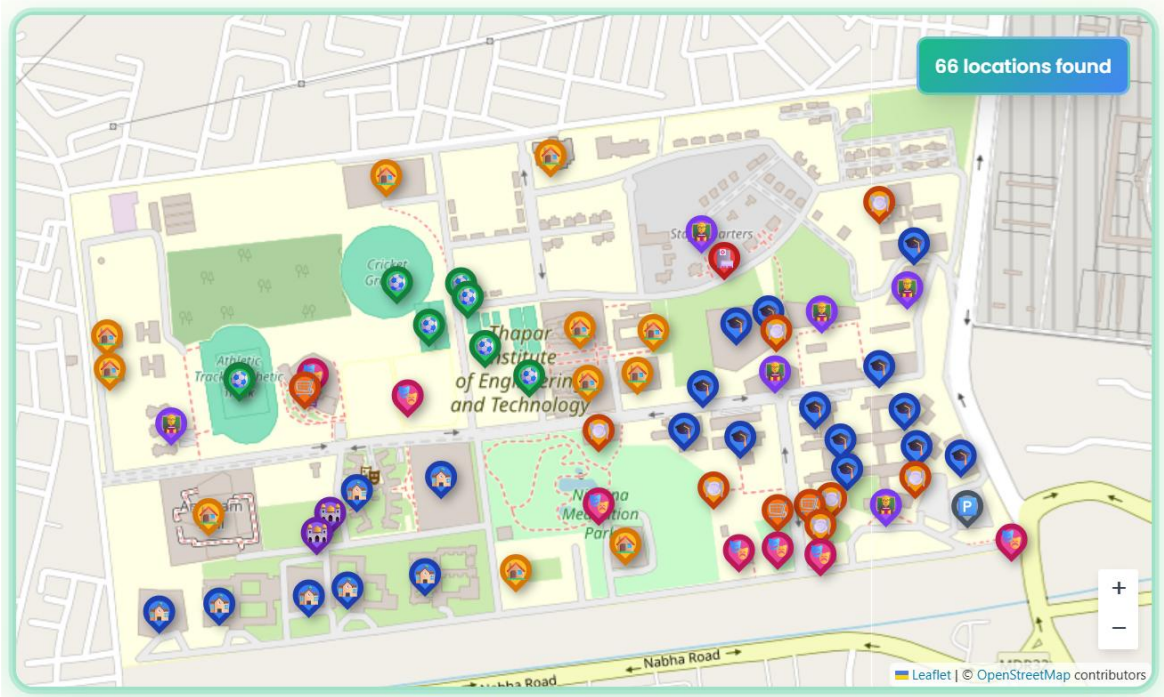


4.3 Screenshots

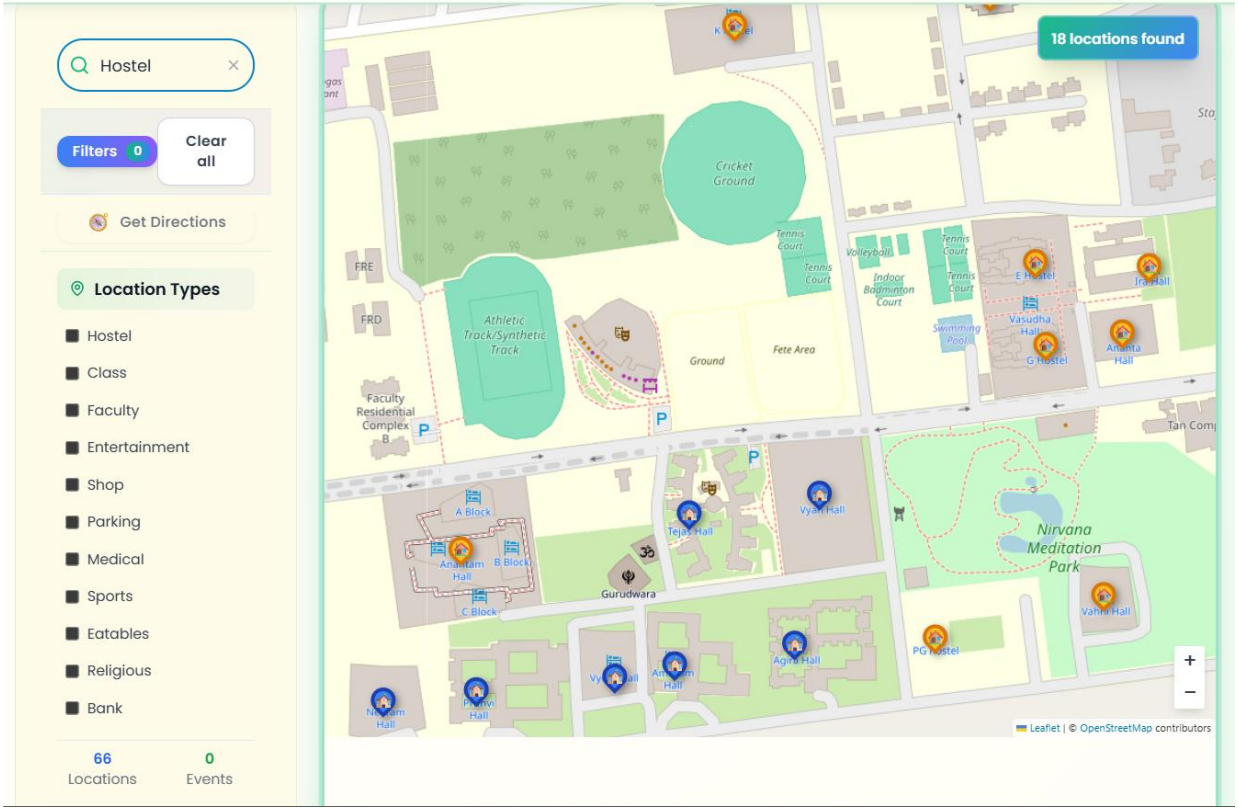
User Registration:



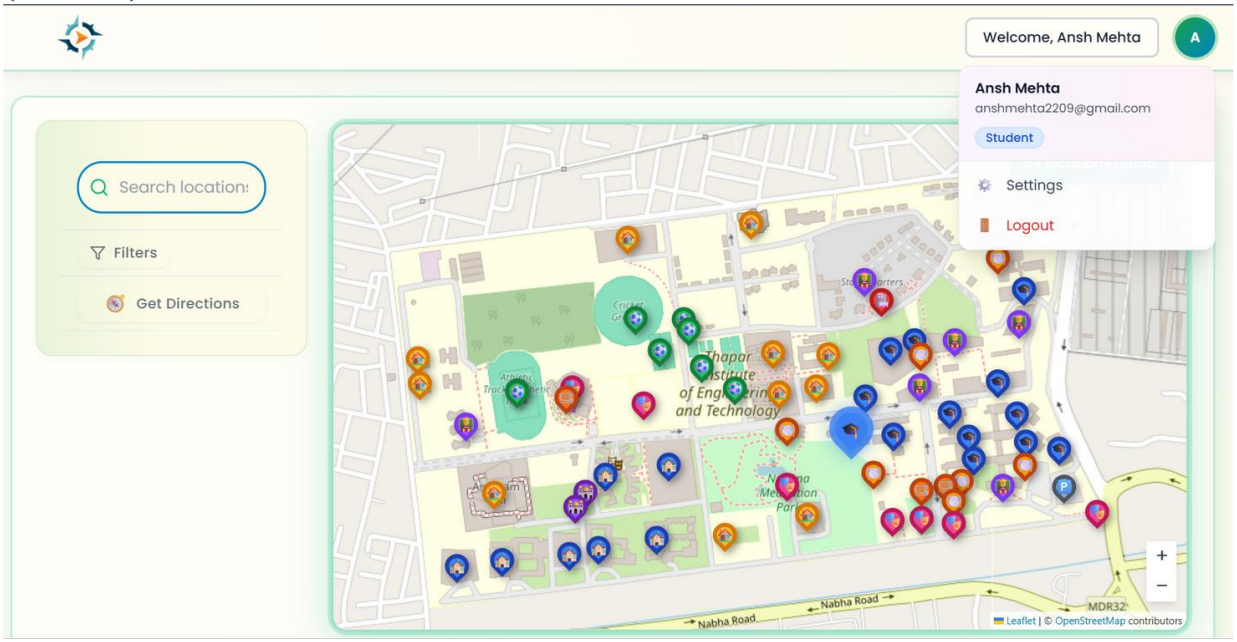
Interactive Campus Mapping:



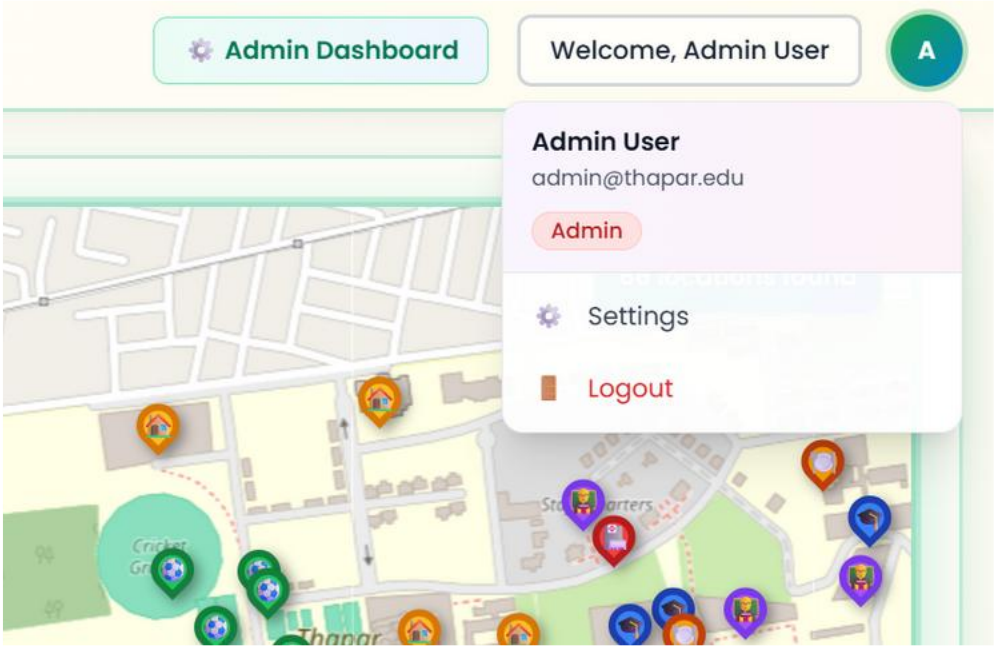
Advanced Search and Filtering:

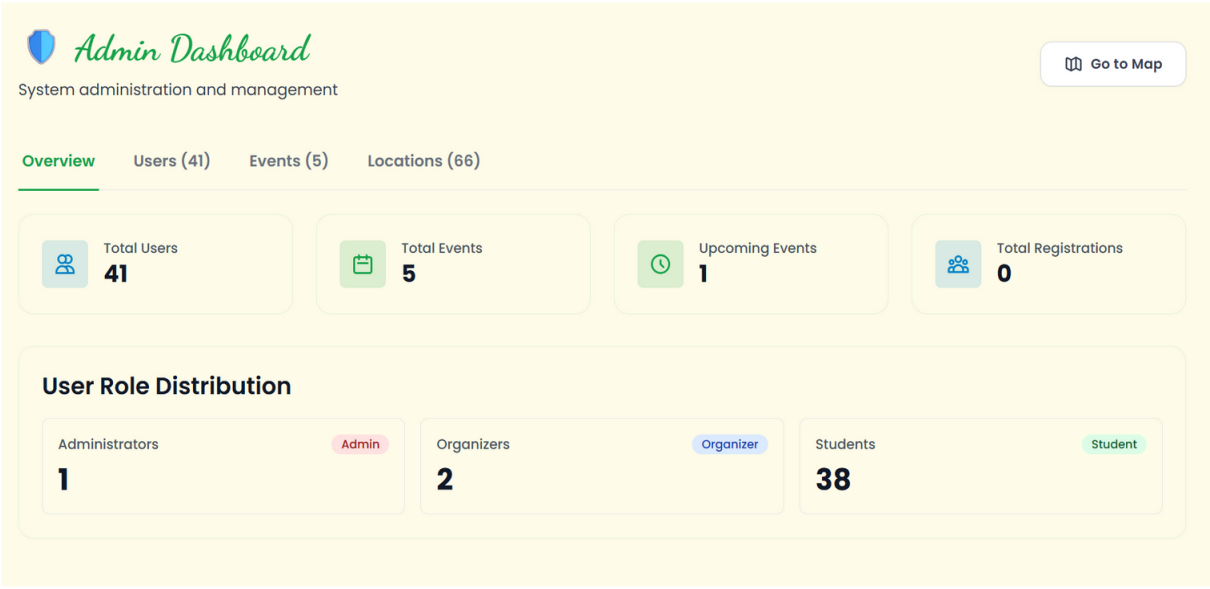


User Authorization:
(Student)

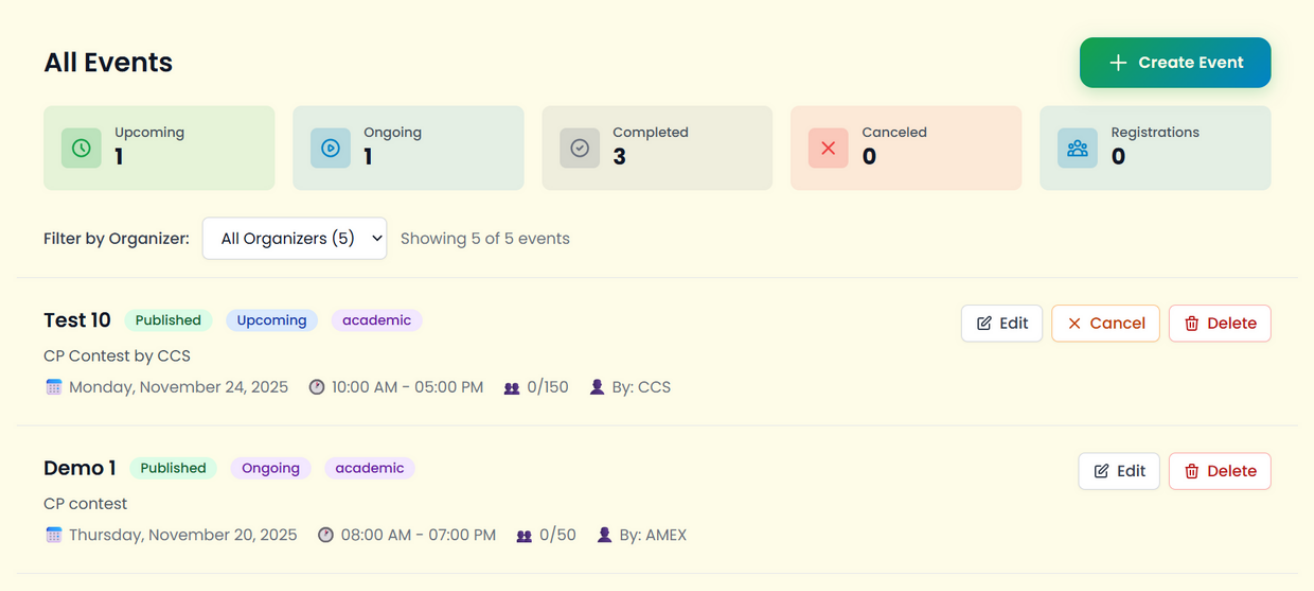


(Admin)

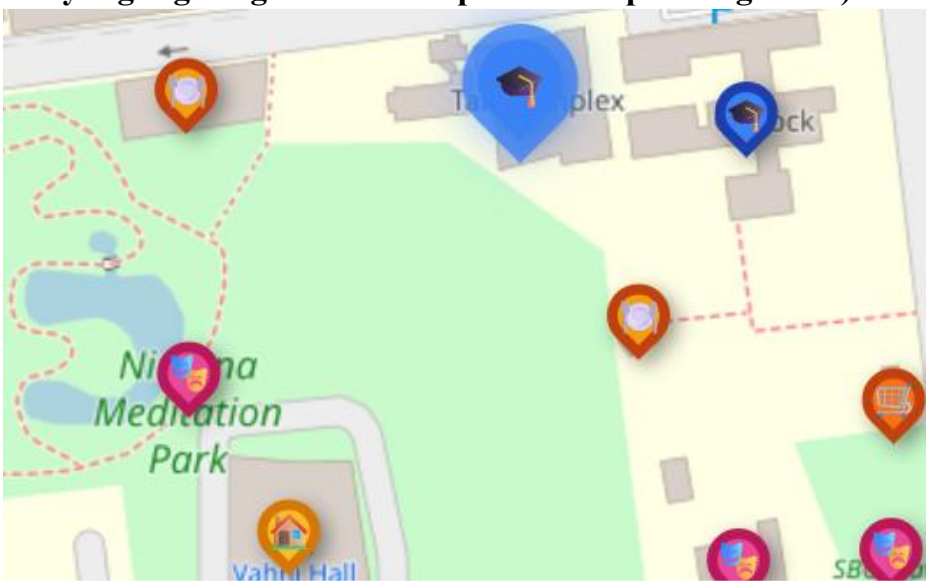

Admin Dashboard
Welcome, Admin User
A
Admin User
admin@thapar.edu
Admin
Settings
Logout


Admin Dashboard
System administration and management
Go to Map
Overview
Users (41)
Events (5)
Locations (66)
Total Users
41
Total Events
5
Upcoming Events
1
Total Registrations
0
User Role Distribution
Administrators
Admin
1
Organizers
Organizer
2
Students
Student
38

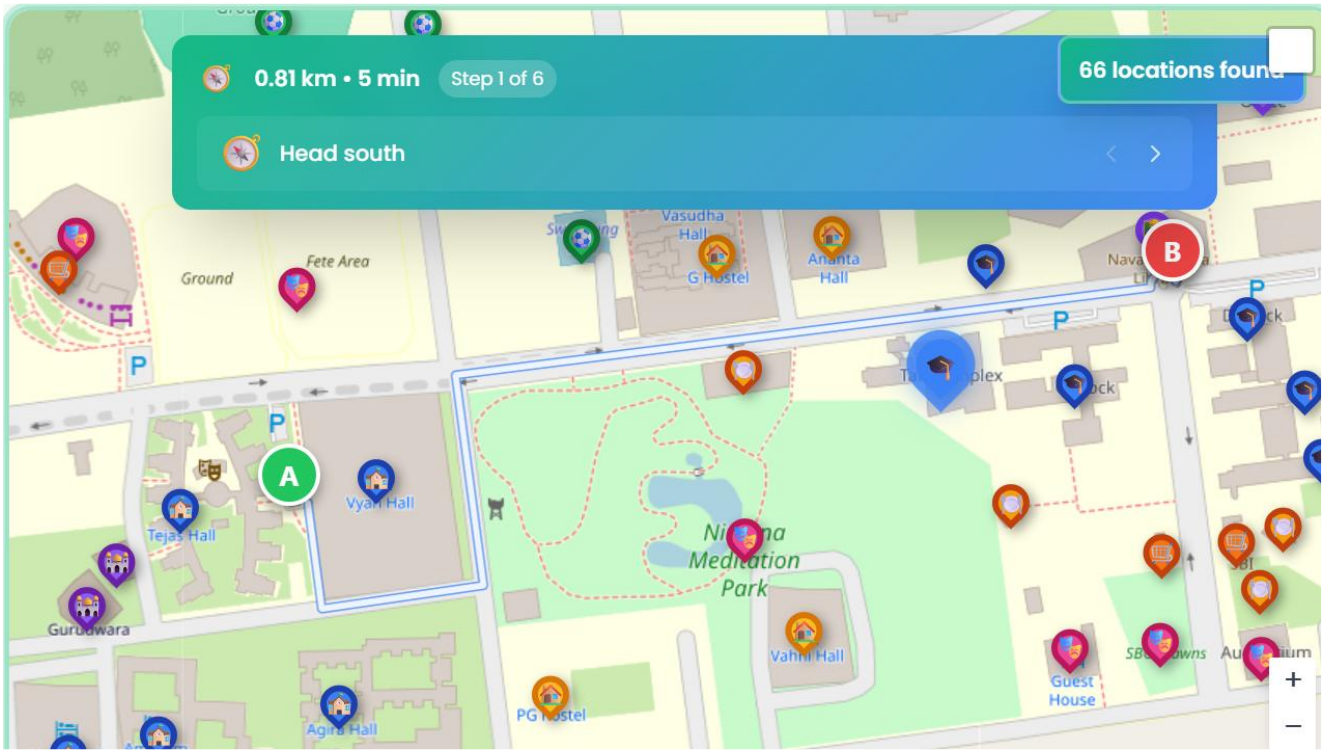
Event Hub:
(Admin/Organizer View for Event Management)



(Event marked by highlighting the location pin of the upcoming event)



Path Finder:



5. Testing

5.1 Test Plan

A test plan outlines the overall strategy and objectives for testing the SmartNav application. It defines what features and functionalities will be tested, the types of testing to be performed (such as unit, integration, and system testing), the resources required, the testing schedule, and the criteria for success. The test plan ensures that all critical components of the application, including user authentication, event management, map navigation, and data storage, are thoroughly validated to meet quality standards and user requirements. By following a structured test plan, the team can systematically identify and address defects, ensuring a reliable and robust final product.

5.2 Test Case

Test cases are specific scenarios designed to verify that individual features and functionalities of the application work as intended. Each test case includes a description of the feature to be tested, the steps to execute, the expected outcome, and the actual result. For SmartNav, test cases might cover actions such as user login, event creation, map rendering, and data retrieval from the database. Well-defined test cases help ensure comprehensive coverage of the application's functionality and make it easier to detect and fix issues during development.

Test Case #: 1	**Test Case Name: User Login with Google**	**Page: 1 of 1**
System: Smart Nav	**Subsystem: Authentication**	
Designed by: Nitin Gupta	**Design Date: 02/11/2025**	
Executed by: Arpan Goyal	**Execution Date: 03/11/2025**	
Short Description: Test the Google Sign-In authentication process.		

Pre-conditions: User is on the login page; Google account exists.

Step	Action	Expected System Response	Pass/ Fail	Comment
1	Click the “Sign in with Google” button	The system displays a Google Login popup.		
2	Select a valid Google account	The system authenticates and redirects to the dashboard/home page.		
3	Check post-condition	The user is logged in and session is active.		

Post-conditions : The user is logged in and can access protected features

Test Case #: 2	**Test Case Name: Add New Event**	**Page: 1 of 1**
System: SmartNav	**Subsystem: Event Management**	
Designed by: Nitin Gupta	**Design Date: 03/11/2025**	
Executed by: Arpan Goyal	**Execution Date: 04/11/2025**	
Short Description:		

Pre-conditions : The user is logged in as an Organizer/Admin and is on the Add Event page.

Step	Action	Expected System Response	Pass/ Fail	Comment
1	Fill in event	The system accepts the input in the form fields.		
2	Click “Submit”	The system validates and creates the event.		
3	Check post-condition	The event appears on the map and in the event list.		

Post-conditions : The new event is saved in the database and visible to users.

Test Case #: 3	**Test Case Name: Map Navigation Route Display**	**Page: 1 of 1**
System: SmartNav	**Subsystem: Map Navigation**	
Designed by: Nitin Gupta	**Design Date: 04/11/2025**	
Executed by: Arpan Goyal	**Execution Date: 05/11/2025**	
Short Description: Test the display of navigation routes on the map		

Pre-conditions : The user is logged in and the map is loaded.

Step	Action	Expected System Response	Pass/ Fail	Comment
1	Select start and end locations	The system highlights the selected points on the map		
2	Click “Show Route”	The system displays the shortest route and step-by-step directions.		
3	Check post-condition	The route remains visible until cleared or changed.		

Post-conditions : The route is displayed on the map for the selected locations.

Test Case #: 4	**Test Case Name: Unauthorized Event Creation Attempt**	**Page: 1 of 1**
System: SmartNav	**Subsystem: Access Control**	
Designed by: Nitin Gupta	**Design Date: 05/11/2025**	
Executed by: Arpan Goyal	**Execution Date: 06/11/2025**	
Short Description: Test that regular users (Students) cannot access event creation.		

Pre-conditions: The user is logged in as regular user (not Organizer/Admin)

Step	Action	Expected System Response	Pass/ Fail	Comment
1	Try to access "Add Event" page	The system denies access and shows an error or warning message.		
2	Check post-condition	The user remains on the current page or is redirected.		

Post-conditions : The user is not able to create an event and receive an appropriate mesaage.

5.3 Test Reports by Peers

Test reports by peers document the results of testing performed by team members or external reviewers. These reports include details about which test cases were executed, any defects or issues found, and feedback on the usability and performance of the application. Peer testing provides an additional layer of quality assurance by bringing in fresh perspectives and helping to uncover problems that the original developers might have missed. The feedback collected through peer test reports is invaluable for improving the overall quality and user experience of the SmartNav application.

To evaluate the usability and performance of the SmartNav system, a **Google Form–based feedback survey(https://forms.gle/nrztCVV7sSF6ru5k9)** was conducted among beta testers. The participants interacted with the deployed website and provided responses regarding layout intuitiveness, responsiveness, functionality, and overall user experience.

Key Findings:

- **Navigation Layout:**
  On a scale of 1–5, the average rating for layout intuitiveness was **4.2**, indicating that users found the interface clear and easy to understand.
- **Ease of Use:**
  Most users reported being able to find the desired information easily without confusion or unnecessary clicks.
- **Responsiveness:**
  The majority mentioned that **SmartNav adapted well to different screen sizes**, with only minor alignment issues on mobile for some users.
- **Performance:**
  4 out of 5 users reported **no lag or delay** during interaction, showing smooth transitions and animations.
- **Navigation Functionality:**
  The "active page" indicator worked correctly for most testers, ensuring good orientation while navigating through the website.
- **Accessibility:**
  Around 60% of users could navigate using the keyboard (Tab/Enter), indicating partial accessibility support.
- **Readability:**
  All users agreed that **text contrast** was sufficient for comfortable reading.
- **Bugs:**
  No major bugs were encountered; minor observations were noted but not reproducible.
- **Suggestions:**
  Some users suggested enhancing animations and adding more interactive elements to improve user engagement.

Conclusion:

The overall feedback reflects a **positive user experience**, with smooth functionality and intuitive design. Testers found the system reliable and visually appealing. Future improvements will focus on **mobile optimization**, **live step-by-step navigation**, and **minor UI enhancements**.